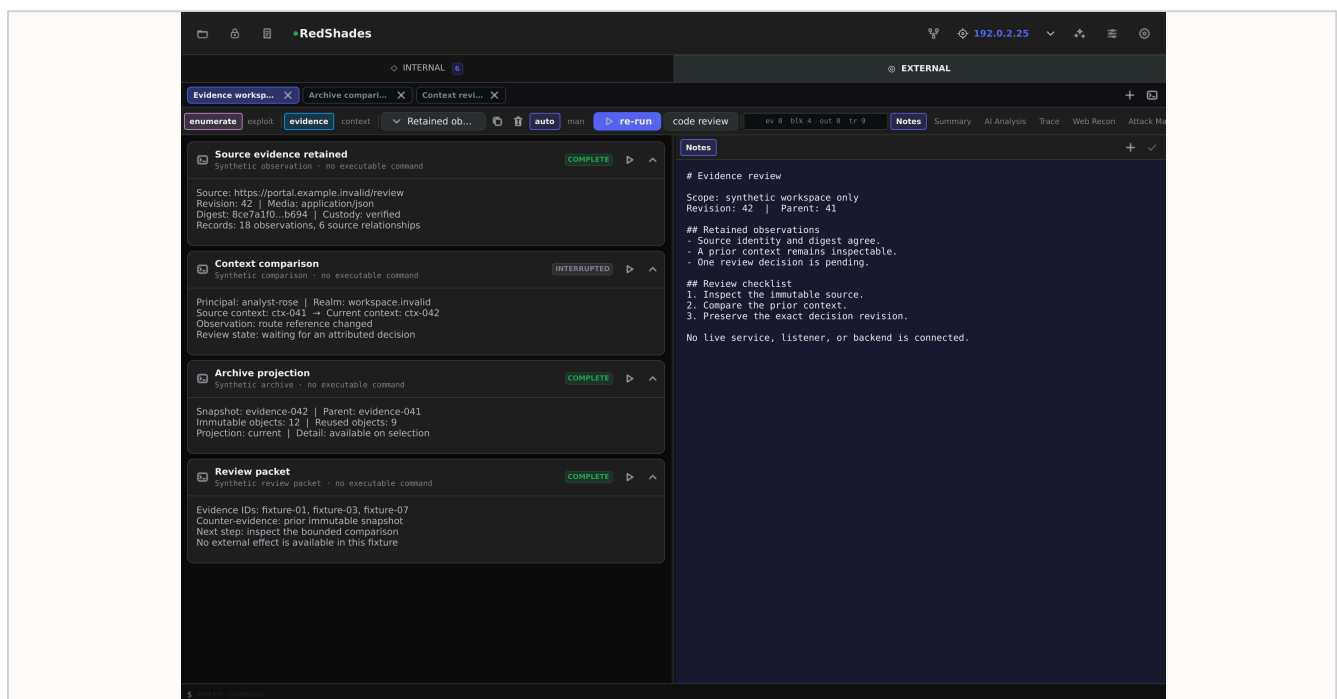


THE PRODUCT AND ENGINEERING PROPOSITION

RedShades. Intelligence with an operating system.

An 88-stage adaptive chain, custom capability generation, mid-run software repair and a native Qt workspace connect in one independently developed platform. RedShades combines deterministic orchestration with focused AI reasoning to carry complex security investigations across tools, branches and long-running sessions.



Actual Qt capture - 5 September 2026 - fictional, disconnected fixture. Presentation evidence only.

The central proposition is architectural: spend model capacity on interpretation and synthesis while software owns workflow, context, validation and execution policy. The 88 automatic declarations form a conditional graph; separate C2 stage families extend session work.

Implemented contract

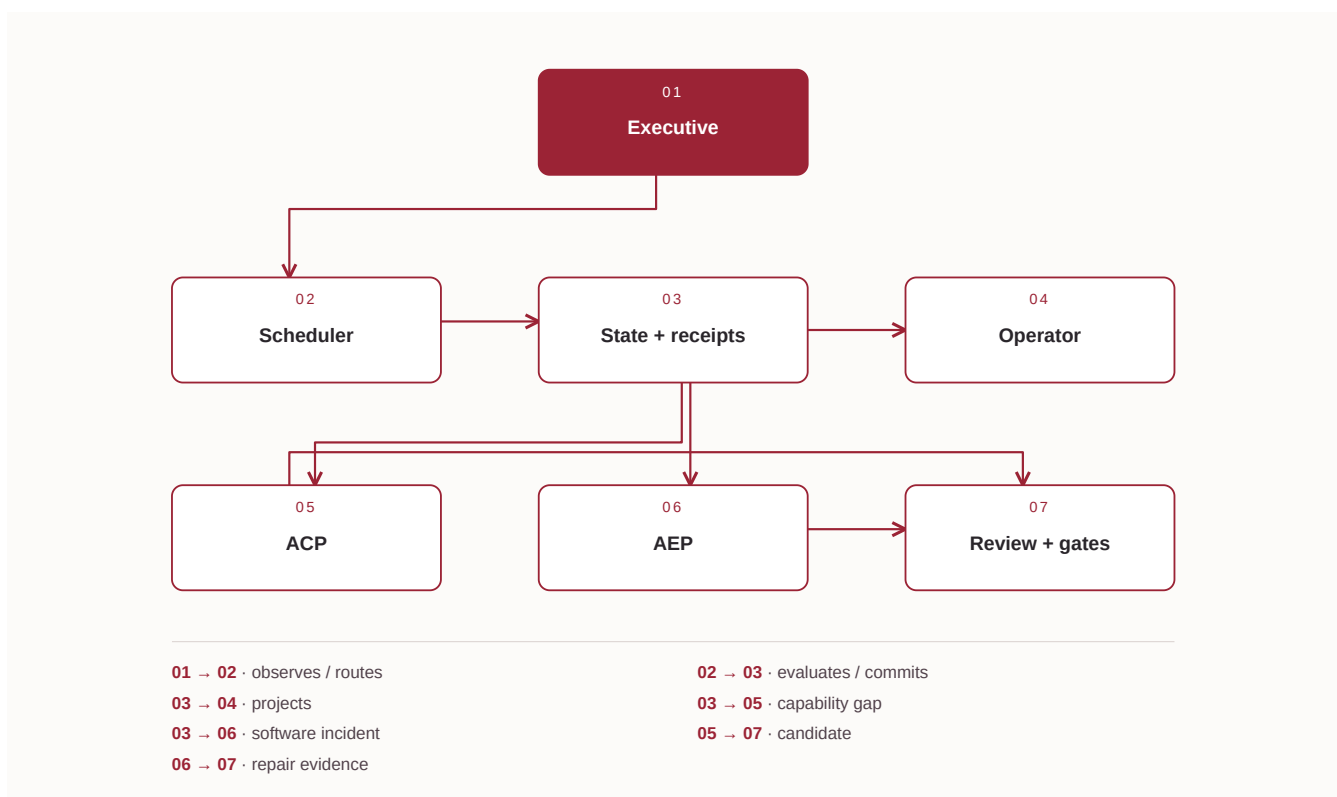
Reference 01

authoritative engagement architecture

WHO DECIDES WHAT HAPPENS NEXT?

A chain that schedules, branches and adapts

The scheduler evaluates 88 automatic stage declarations against current evidence. Independent branches advance when their inputs are ready; loops revisit useful frontiers as observations change. The Executive maintains the wider view while ACP supplies missing capabilities and AEP repairs the underlying software.



Source-derived conceptual diagram. Arrows describe the labeled relationship.

Each role has a distinct responsibility: scheduling, supervision, bounded interpretation, capability generation or platform repair. Their shared records let the investigation progress beyond one prompt or one tool.

Implemented contract

Reference 02
 subsystem glossary

PARENT CONTEXT, CHILD SCOPE, CONNECTED PROGRESS

Pivot into a child investigation

A newly available route can open a child subchain with its own scope and execution context. Parent and child retain their relationship, so a deeper investigation remains part of the same attributed engagement.



Actual Qt capture - 5 September 2026 - fictional, disconnected fixture. Presentation evidence only.

Frontiers track eligible and waiting work. Branch-local decisions allow unrelated work to continue; parent completion accounts for attached investigations. This is a graph of evolving work rather than a fixed checklist.

Implemented contract

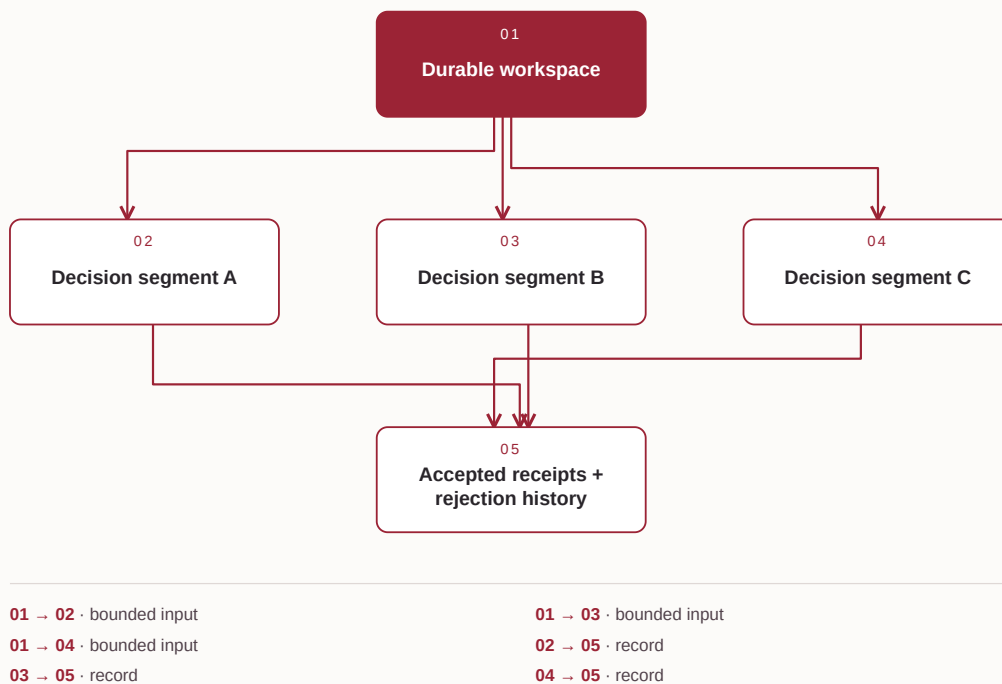
Reference 03

authoritative engagement architecture

HOW AI ALLOCATION CONNECTS TO PRODUCT VALUE

70% deterministic. 30% neural.

RedShades uses this ratio as a design philosophy: dependable software surrounds the places where model judgment adds value. Scheduling, state, policy and validation provide structure; models interpret ambiguity, develop candidates and guide complex engineering work.



Source-derived conceptual diagram. Arrows describe the labeled relationship.

The ratio is a design framing, not a measured code, runtime or success percentage. The intended economic advantage is selective intelligence: compact specialists for bounded decisions, persistent sessions for work that benefits from continuity.

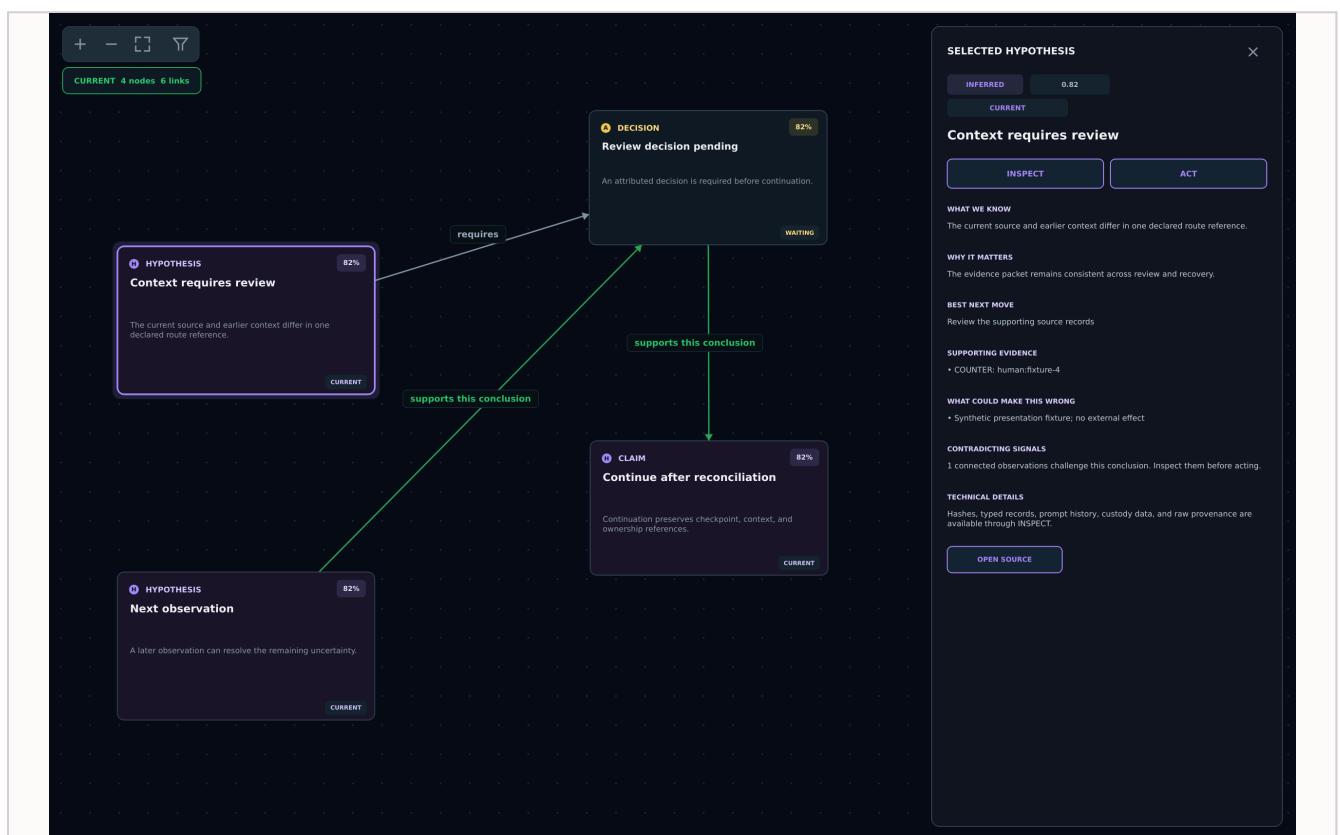
Design philosophy

Reference 04
acp vertical session

WHAT BELONGS TOGETHER?

An engagement memory shared by every subsystem

Evidence, identities, routes, artifacts, capabilities and outcomes are linked in versioned engagement records. The scheduler, model workspaces and operator views can work from the same account of the investigation.



Actual Qt capture - 5 September 2026 - fictional, disconnected fixture. Presentation evidence only.

Revision-bound records preserve the inputs behind each decision. Canonical state supplies the architecture for coordinated work across tools and sessions; source references document the implementation boundaries.

Implemented contract

Reference 05

authoritative engagement architecture

WHAT MAKES EVIDENCE REUSABLE?

From tool output to reusable evidence

Integrated tools contribute observations with their source, context and interpretation retained. Source references make an accepted result useful to the next stage, the next model segment and the operator inspecting the decision.



Actual Qt capture - 5 September 2026 - fictional, disconnected fixture. Presentation evidence only.

Evidence categories support correlation without flattening distinct candidates into one summary. This is the connection between broad tool coverage and coherent automated reasoning.

Implemented contract

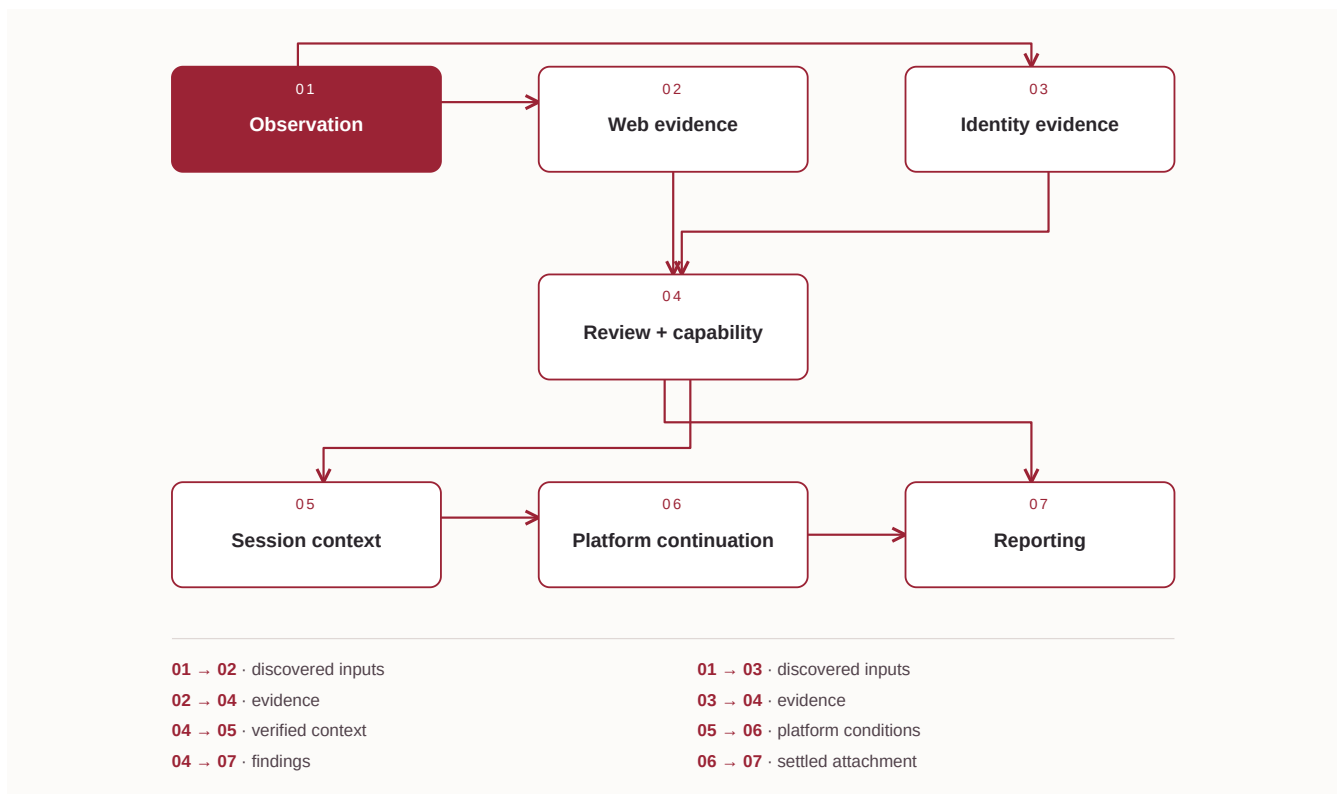
Reference 06

canonical specialist contracts

WHAT DOES THE STAGE COUNT DESCRIBE?

88 automatic stages, one adaptive graph

Discovery, service and identity analysis, web investigation, artifact analysis, session work and reporting are represented in 88 automatic stage declarations. A separate set of 13 C2 declarations adds common and platform-specific session families.



Source-derived conceptual diagram. Arrows describe the labeled relationship.

Prerequisites, evidence, platform selection and operator policy select the actual path. The stage atlas provides every automatic declaration with its source label and input/output role; 88 does not describe a mandatory linear execution sequence.

Implemented contract

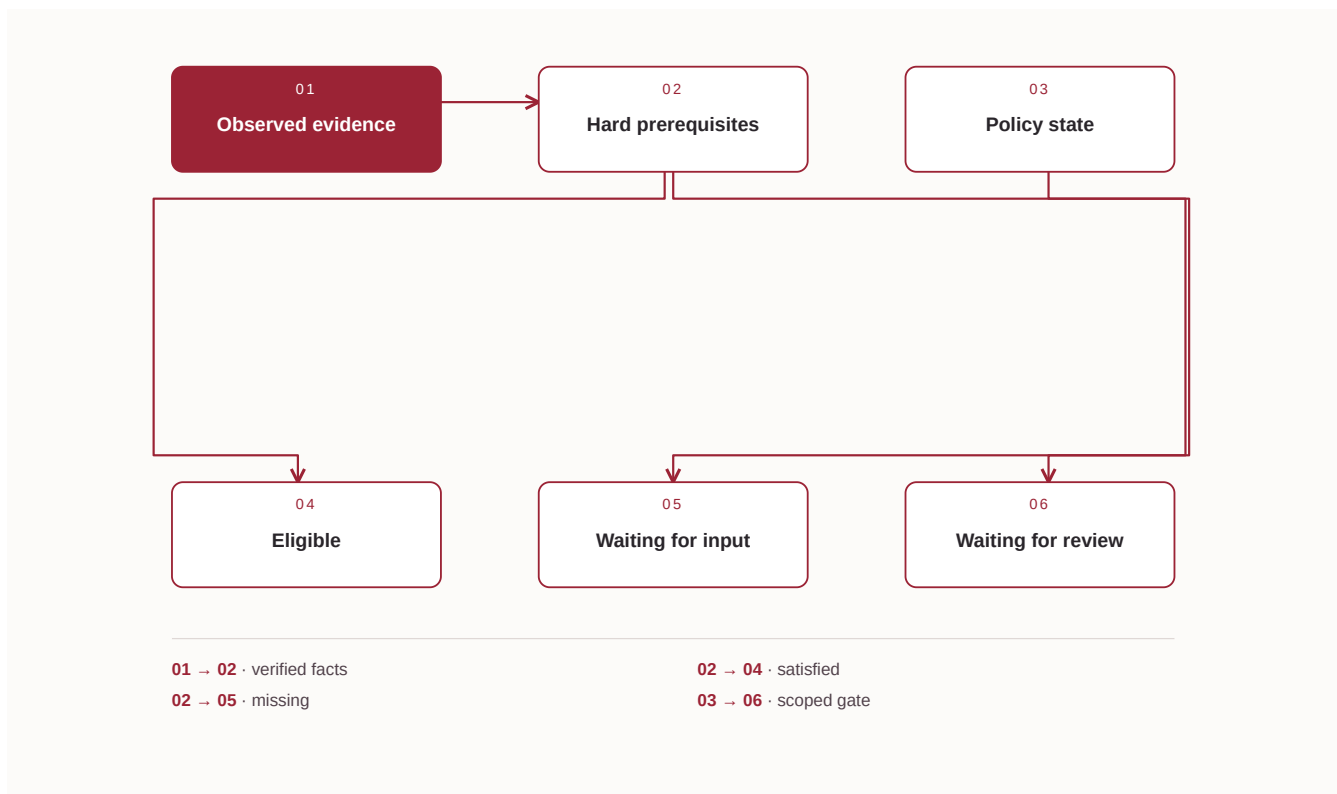
Reference 07

`semantic_registry.go + chain_runtime.go`

HOW THE GRAPH AVOIDS INVENTING AUTHORITY

A scheduler that understands relationships

The Execution Graph distinguishes scheduling dependencies from evidence correlations. Phase barriers, declared branch dependencies and scoped eligibility control work, while context and evidence links enrich the operator's view.



Source-derived conceptual diagram. Arrows describe the labeled relationship.

The scheduler evaluates a bounded stage view. Dynamic endpoint and gate nodes remain useful to the operator without multiplying the work required to decide which stage is ready.

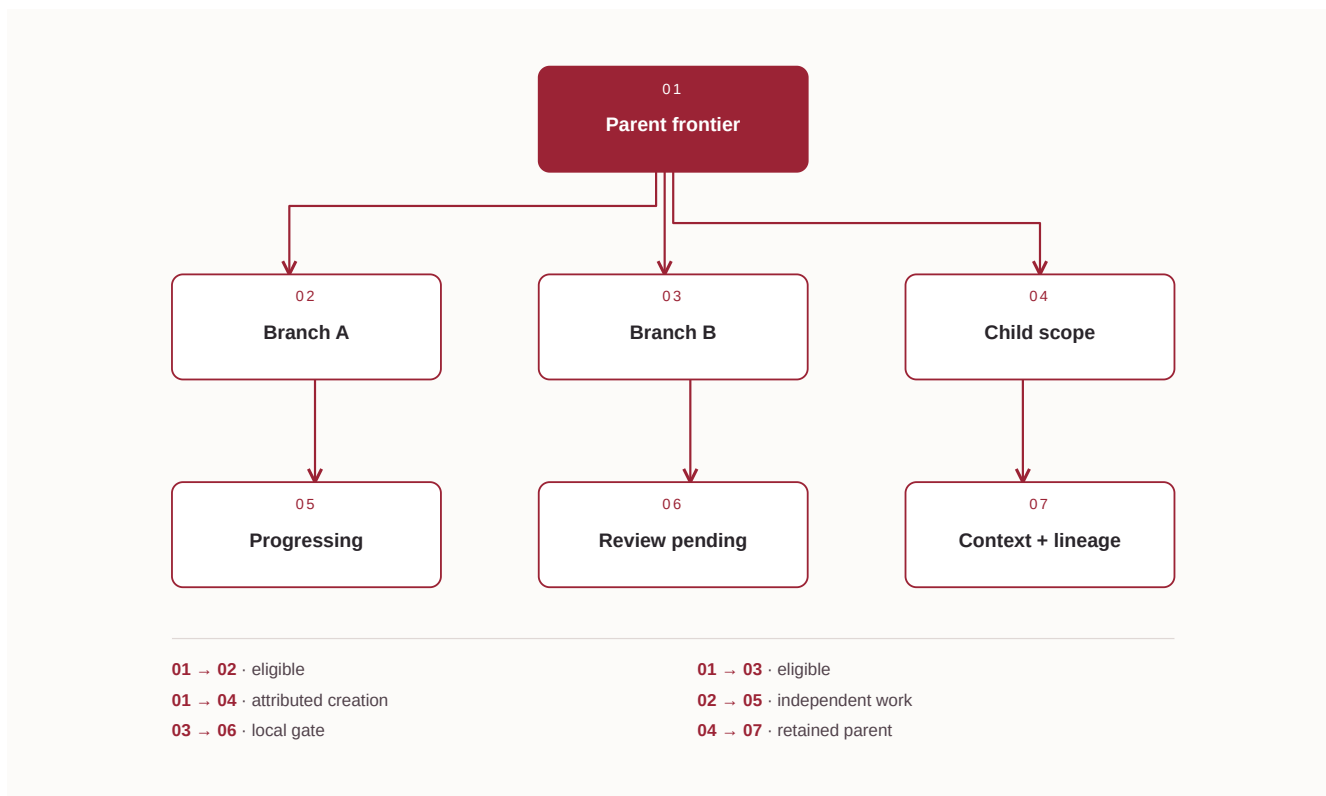
Implemented contract

Reference 08
execution graph lifecycle

HOW BRANCHES PROGRESS INDEPENDENTLY

Keep independent branches moving

The scheduler separates branch-local gates from a global pause. A web review, identity question or pending prerequisite can wait in its own scope while other eligible work advances.



Source-derived conceptual diagram. Arrows describe the labeled relationship.

Loops and dormant branches respond to new evidence. The frontier records which input can change eligibility, giving long investigations a clear continuation point.

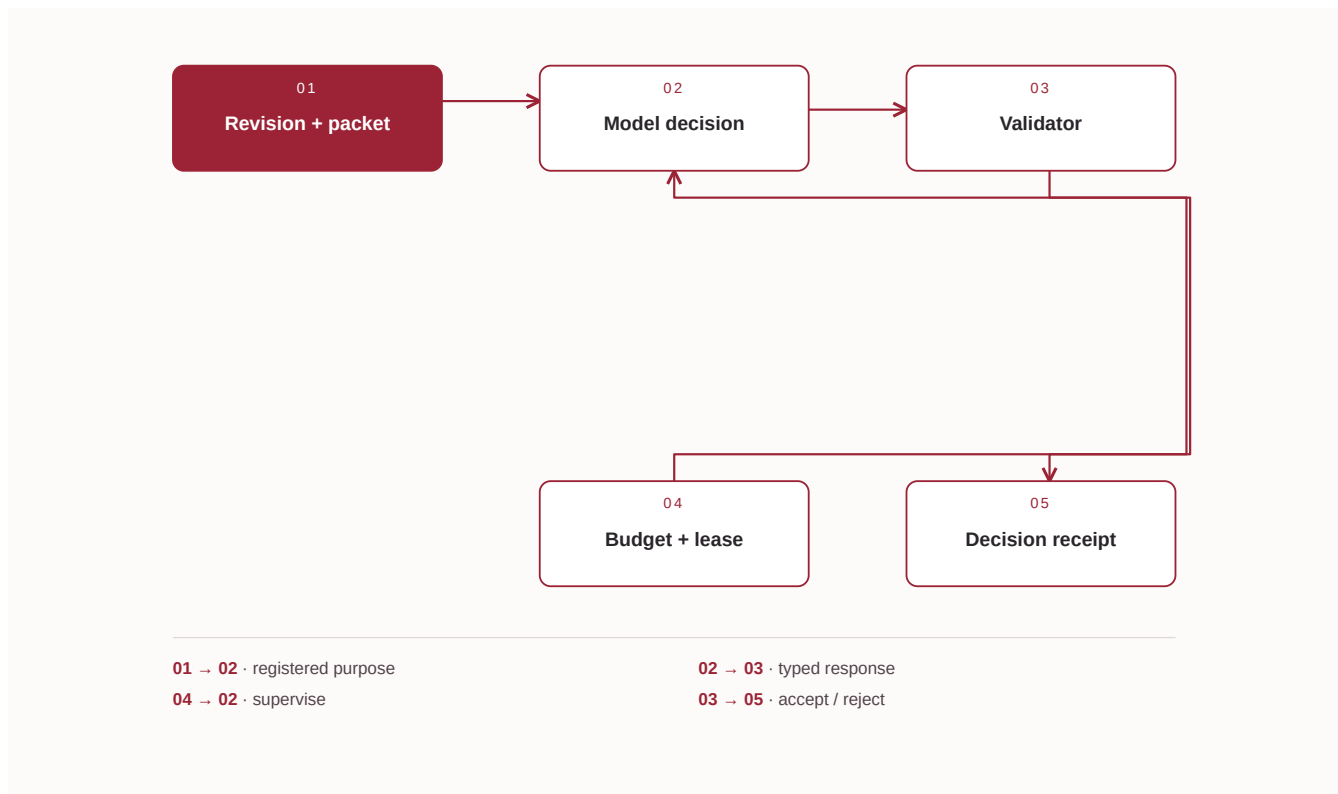
Implemented contract

Reference 09
execution graph lifecycle

WHAT SURROUNDS A DECISION?

Small calls with complete jobs

Bounded stateless AI calls receive a defined purpose and a structured evidence packet. Deterministic code checks the response, attributes the model and retains the accepted result. This is the intended home for compact, fine-tuned specialist models.



Source-derived conceptual diagram. Arrows describe the labeled relationship.

Token accounting, capacity controls and response validation surround the call. A 1B-class specialist is a deployment objective for suitable bounded tasks; the catalog does not claim a trained model bank or measured equivalence to a larger model.

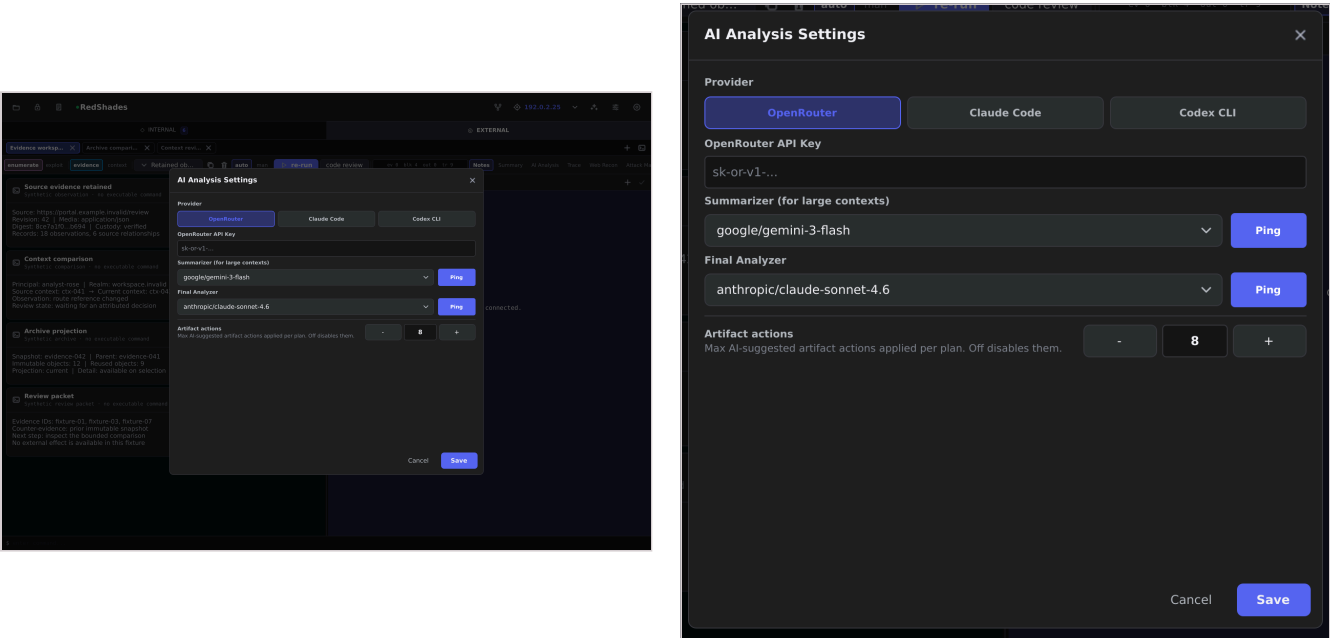
Implemented contract

Reference 10
authoritative engagement architecture

WHAT IS THE EXECUTIVE WATCHING?

An Executive that retains the wider mission

The Executive observes progress across stages, prioritizes attention and routes capability gaps or engineering incidents. Its persistent supervisory context complements the short, focused calls used inside individual boundaries.



Actual Qt capture - 5 September 2026 - fictional, disconnected fixture. Overview and detail crop from identical source pixels. Presentation evidence only.

Executive, ACP and AEP workspaces preserve artifacts and checkpoints for longer tasks. Model capacity can be allocated by responsibility, while the deterministic controller retains authority over the workflow.

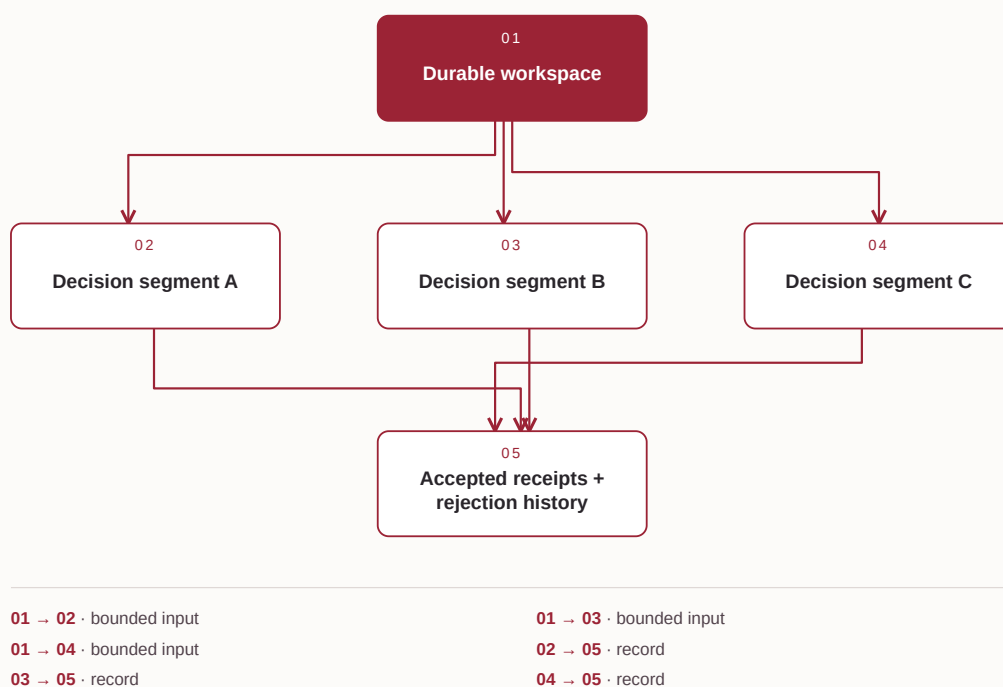
Implemented contract

Reference 11
executive teacher babysitter plan

HOW COULD A COMPACT MODEL CONTRIBUTE?

Compact models, durable engineering context

Persistent Executive, ACP and AEP sessions can use more capable compact models for synthesis and repair. Custom context management carries accepted artifacts, prior review and progress records between segments, so the task does not depend on an indefinitely growing conversation.



Source-derived conceptual diagram. Arrows describe the labeled relationship.

Model selection is configurable. Terra-low is a documented operating policy in the project history; alternative model deployments require their own evaluation. The architecture targets strong outcomes from efficient models without asserting an unmeasured quality or speed equivalence.

Implemented model boundaries

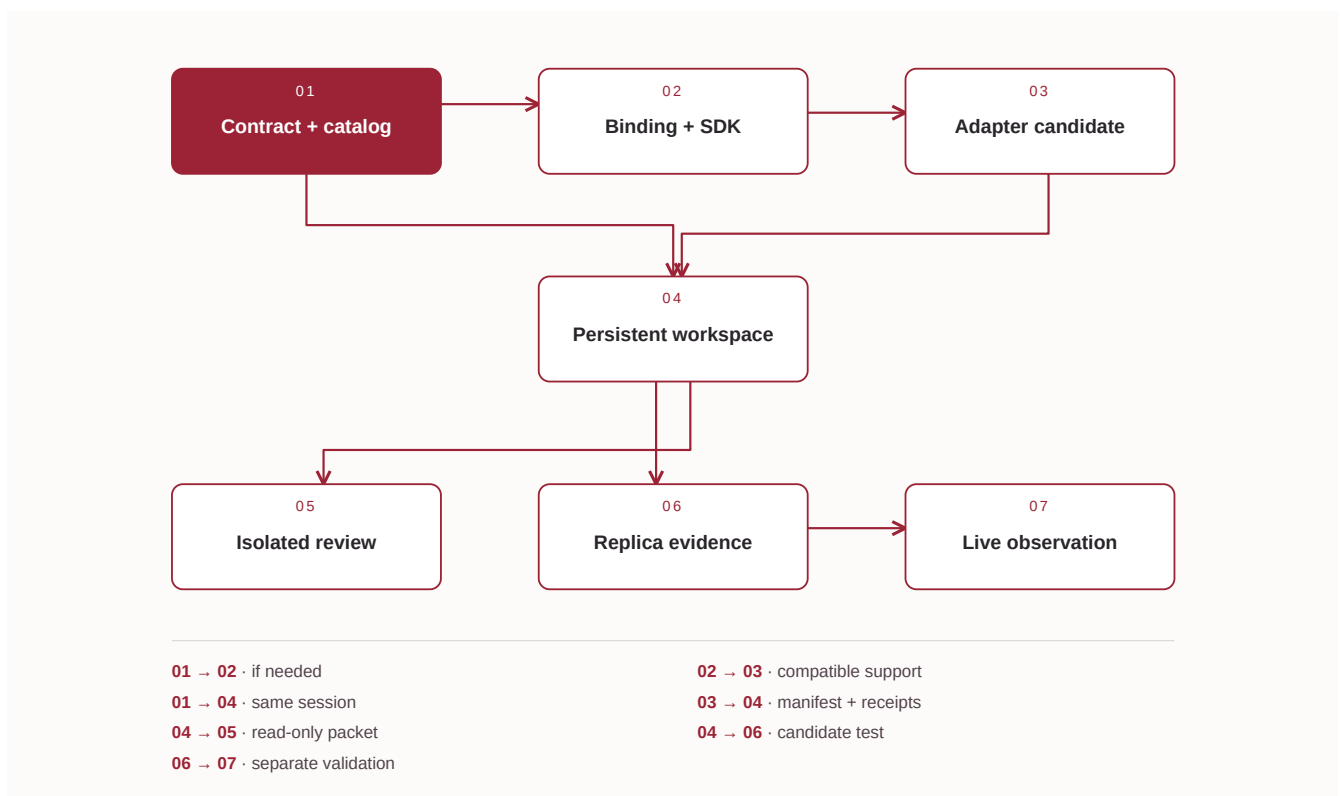
Reference 12

canonical specialist contracts

WHY PERSIST A GENERATION SESSION?

ACP builds evidence-bound capabilities

The Adaptive Capability Pipeline develops custom capability candidates from observed evidence and source mechanics. A persistent workspace owns binding, SDK selection or extension, adapter generation, review and replica validation.



Source-derived conceptual diagram. Arrows describe the labeled relationship.

Compatible catalog artifacts and SDKs can skip generation segments. New work retains its contract and review history, connecting flexibility with reusable, reviewed artifacts.

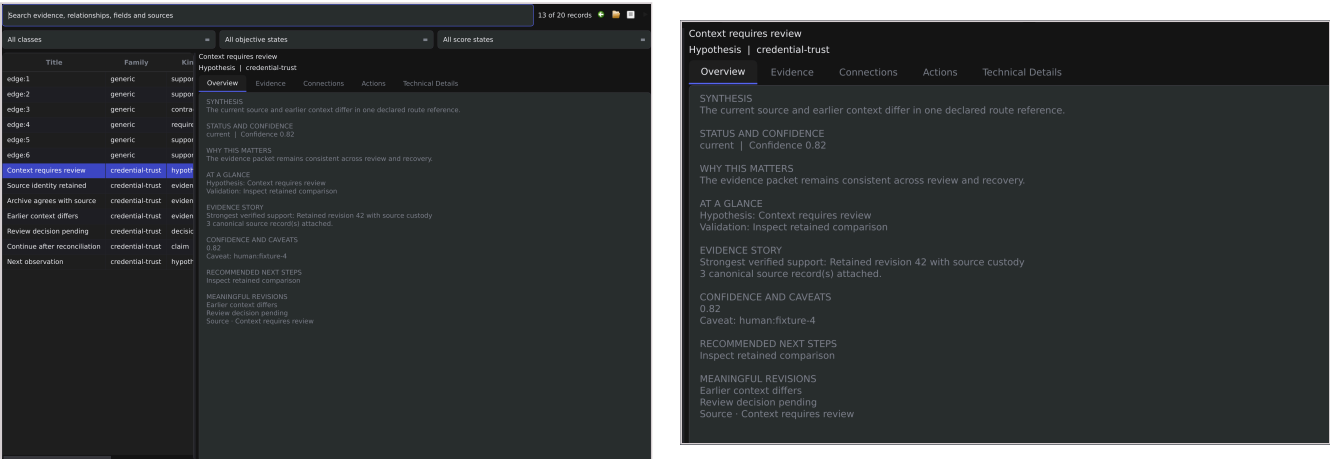
Implemented contract

Reference 13
 acp vertical session

WHAT CROSSES THE GENERATION BOUNDARY?

From an advisory to a reviewed candidate

ACP connects source findings and advisories with typed capability contracts, SDK extensions and reviewed adapters. Where a ready-made proof of concept is absent, the workbench can retain a candidate for controlled evaluation.



Actual Qt capture - 5 September 2026 - fictional, disconnected fixture. Overview and detail crop from identical source pixels. Presentation evidence only.

Candidate source, metadata and tests are published as a digest-bound set.
Admission and isolated review establish the boundary between generated material and an approved artifact.

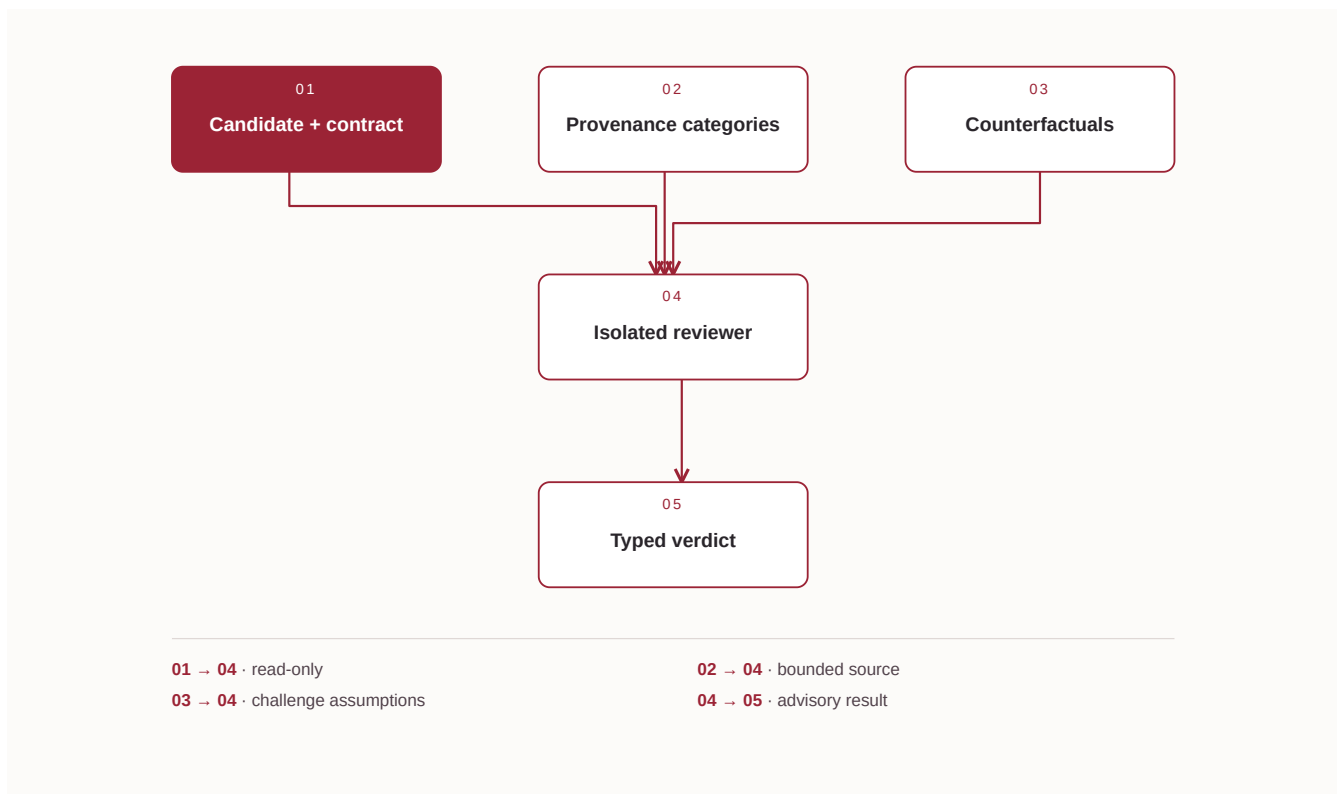
Implemented contract

Reference 14
acp vertical session

WHY ISOLATE THE CONTAMINATION REVIEWER?

Review is built into capability production

An isolated reviewer evaluates the candidate against its contract and provenance. Counterfactuals test whether changed names, ordering and evidence still support the same capability, helping turn a one-off result into a reusable component.



Source-derived conceptual diagram. Arrows describe the labeled relationship.

The reviewer uses a read-only packet and returns a typed verdict to admission. Generation, review and execution authority remain separately owned.

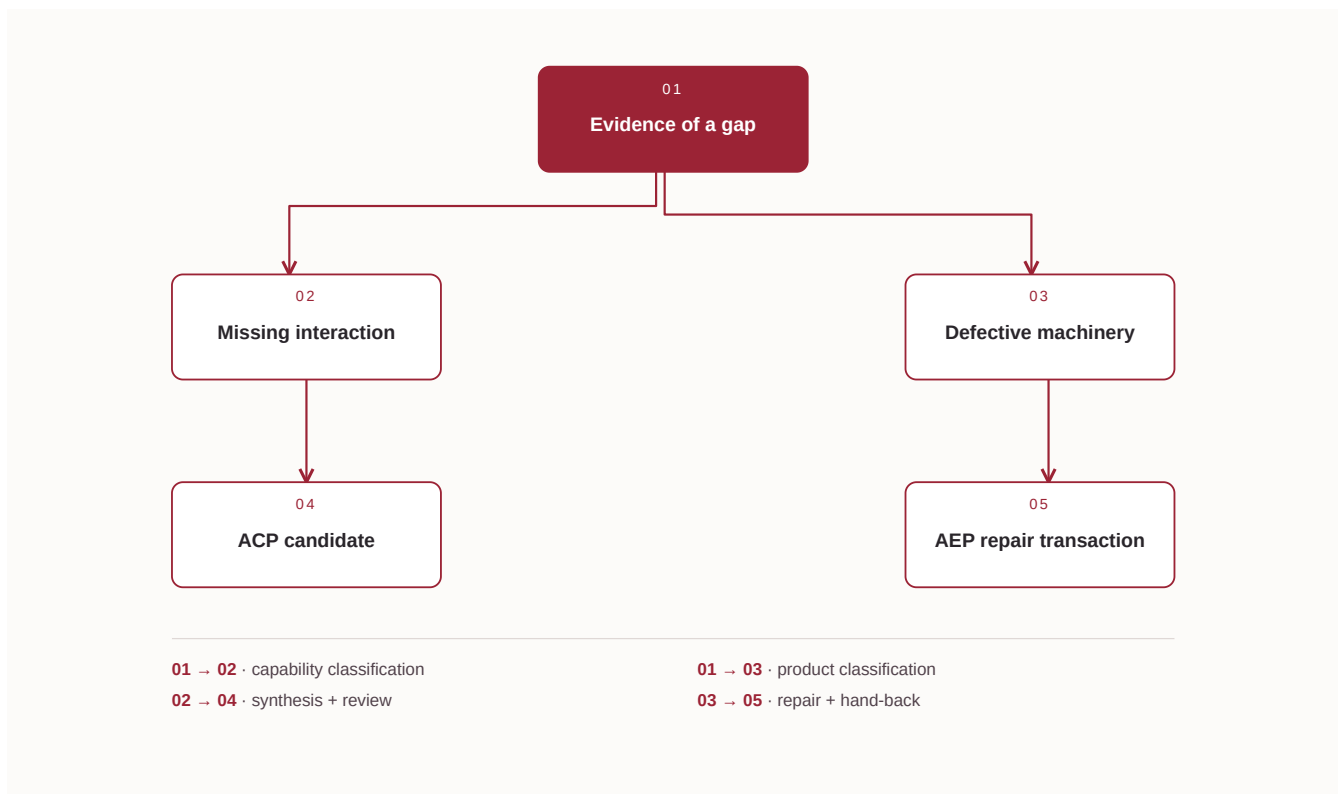
Implemented contract

Reference 15
acp vertical session

WHERE ACP ENDS AND AEP BEGINS

AEP repairs software while the run is in flight

The Autonomic Engineering Plane owns diagnosis, a persistent repair session, scoped source changes, tests, build admission, hot replacement and verified hand-back. It addresses defects in the platform machinery while ACP addresses missing target interactions.



Source-derived conceptual diagram. Arrows describe the labeled relationship.

The full repair transaction preserves the run's context through a provider replacement. Its acceptance is defined by validated hand-back, connecting software engineering work to continued execution.

Implemented contract

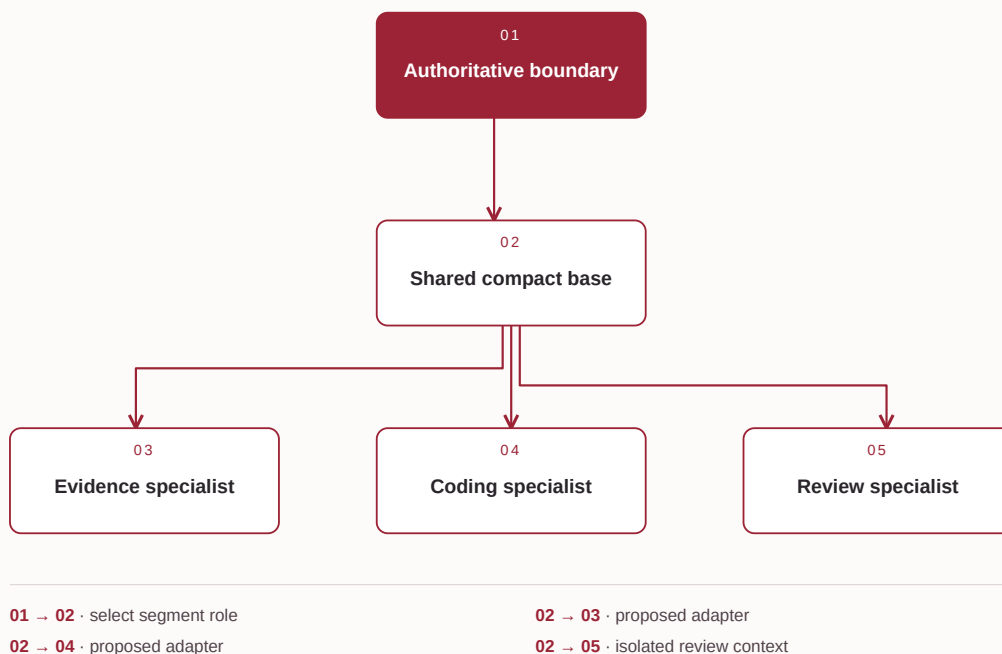
Reference 16

executive teacher babysitter plan

WHAT IS PROPOSED FOR MODEL ALLOCATION?

A proposed bank of 100 specialist adapters

The model direction is a compact shared base with a proposed bank of roughly 100 specialist QLoRA adapters, selected at deterministic task boundaries. Specialized short calls would share the surrounding evidence, workflow and validation architecture.



Source-derived conceptual diagram. Arrows describe the labeled relationship.

This is proposed model architecture, not a claim that the bank is trained or deployed. Persistent synthesis sessions remain a complementary path for harder work; serving and cross-adapter context behavior require evaluation.

Proposed model architecture

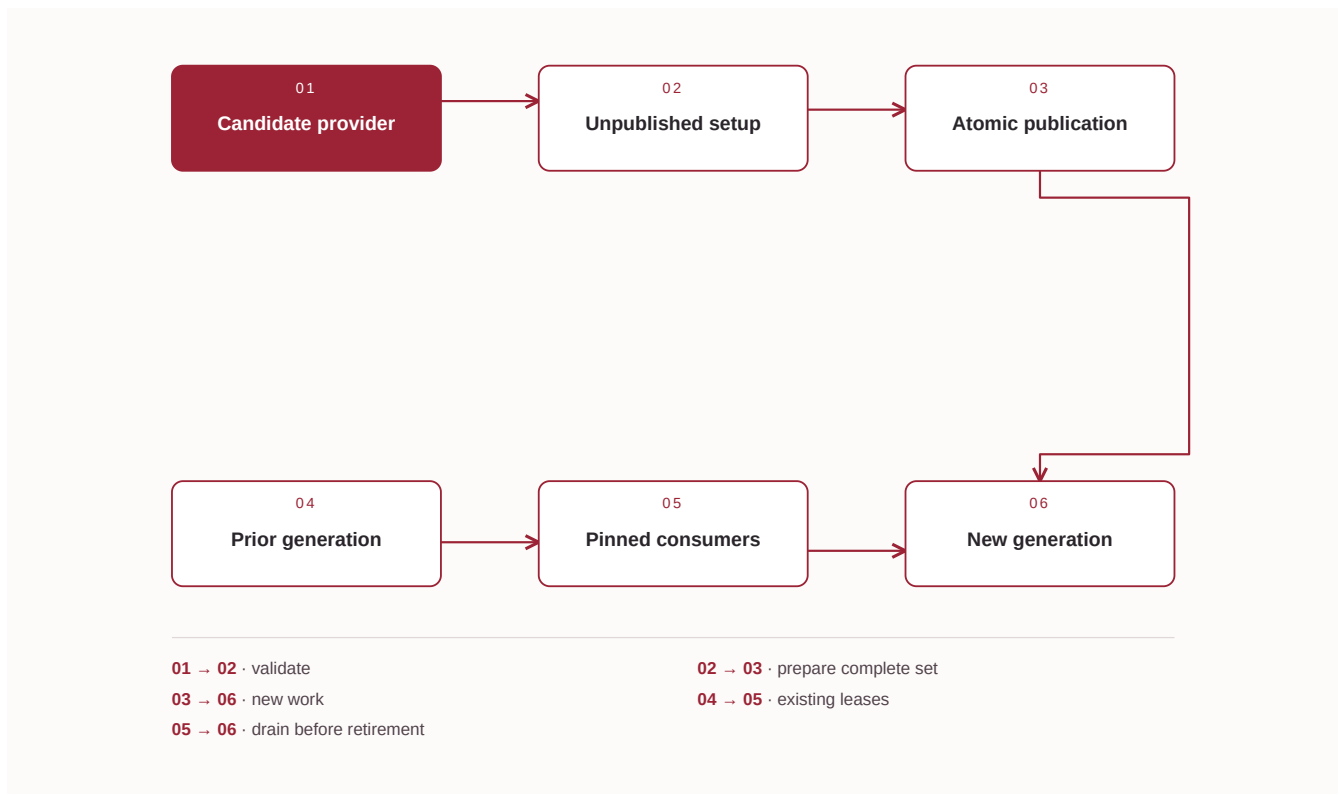
Reference 17

acp vertical session + design history

WHAT DOES THE COMPOSITION KERNEL OWN?

Hot replacement with continuity

The Runtime Composition Kernel publishes versioned providers and controls leases, rollback and cleanup. New work can adopt a replacement while existing work retains its implementation and engagement records.



Source-derived conceptual diagram. Arrows describe the labeled relationship.

This is the runtime foundation beneath AEP hand-back. State remains outside replaceable behavior, giving repair a defined boundary rather than requiring a whole-workspace restart.

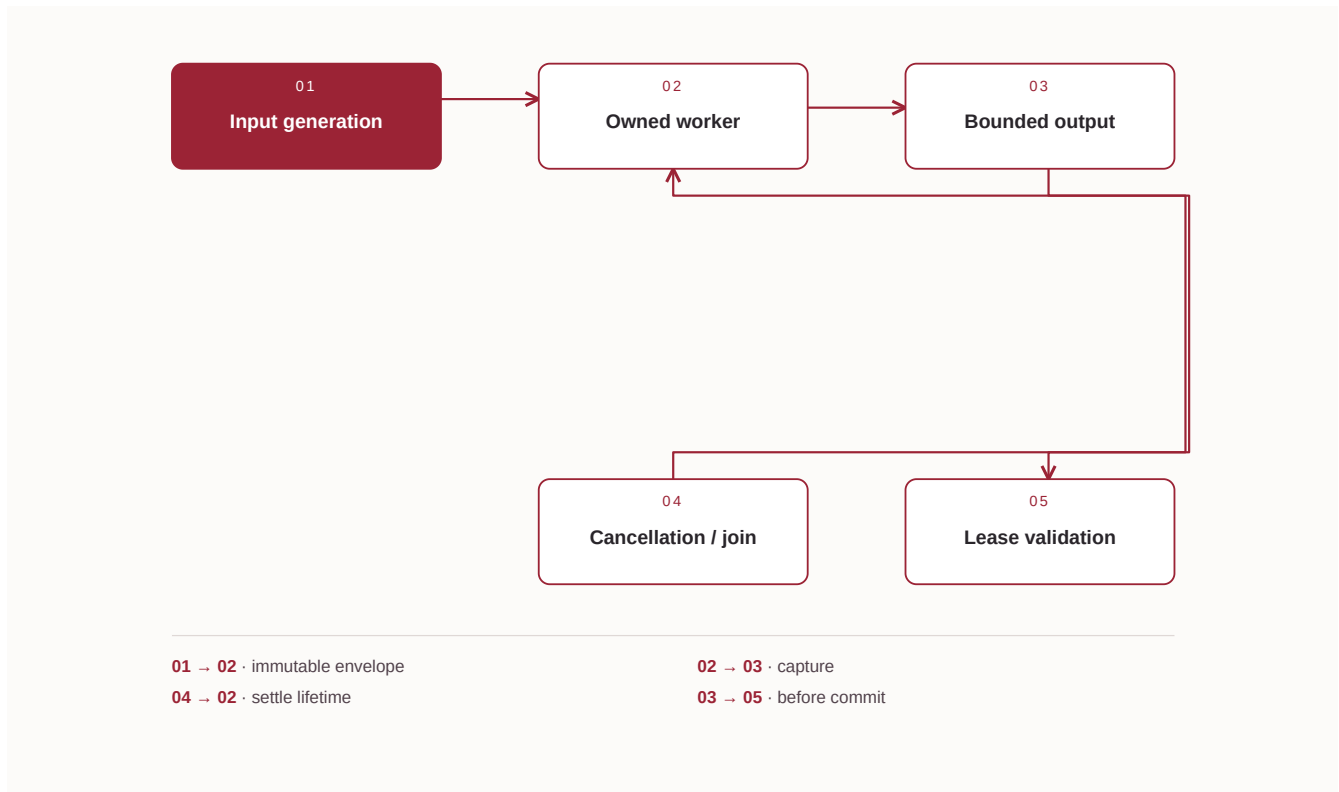
Implemented contract

Reference 18
runtime composition kernel

HOW IS STALE WORK REJECTED?

Workers return results under a lease

Isolated stage workers receive immutable execution envelopes. Results are accepted only against the current artifact, input and generation lease. That prevents an old worker from publishing a plausible result after its authority has expired.



Source-derived conceptual diagram. Arrows describe the labeled relationship.

In-process handlers and isolated workers have different replacement boundaries. The catalog describes the common principle without implying that every component can be replaced at any arbitrary instruction.

Implemented contract

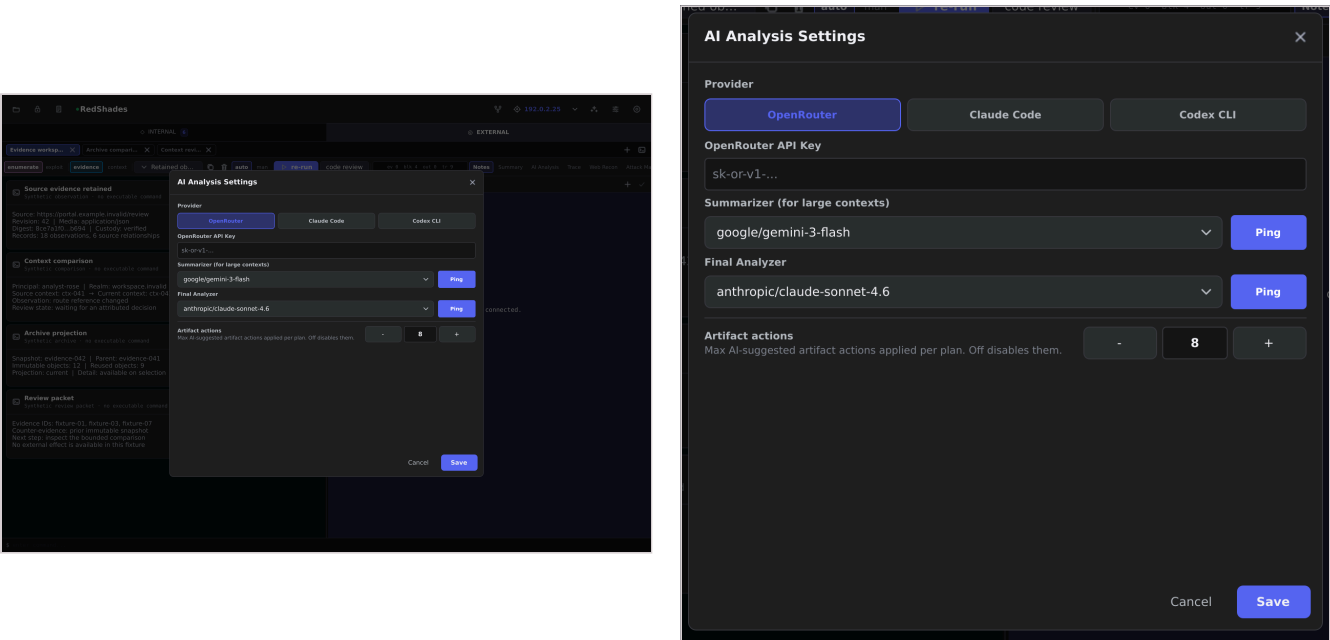
Reference 19

hot swappable stage workers

WHAT DOES CLEANUP NEED TO KNOW?

Context and resource budgets stay visible

Custom context management and token accounting track the cost of model work. Resource supervision retains ownership of worker processes, bounded output and cleanup, supporting long sessions without losing sight of their compute footprint.



Actual Qt capture - 5 September 2026 - fictional, disconnected fixture. Overview and detail crop from identical source pixels. Presentation evidence only.

Token and resource diagnostics make unusual consumption inspectable. Named owners and retained cleanup handles let the platform distinguish active work from stale work.

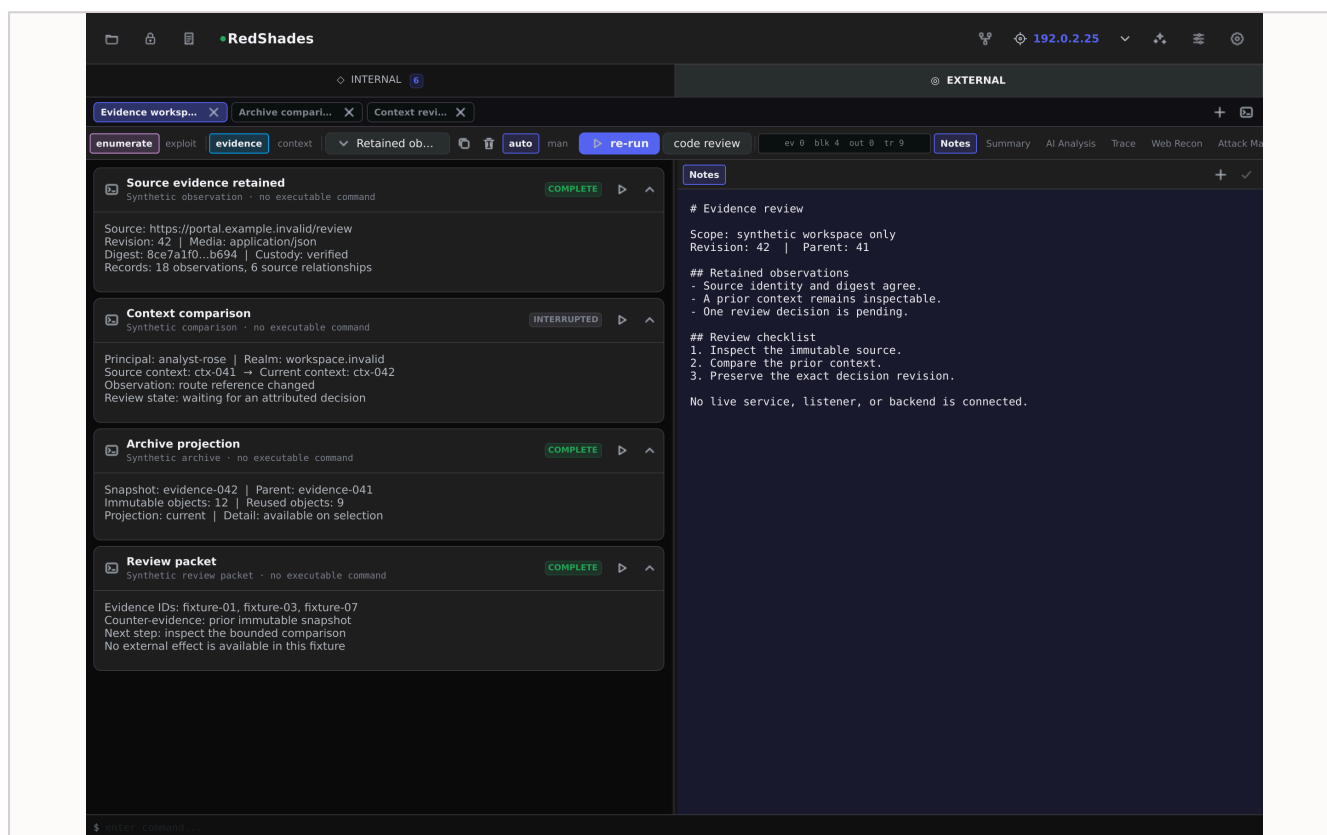
Implemented supervision boundaries

Reference 20
service process custody

HOW THE WORKSPACE FITS TOGETHER

Qt, watchdog and tools in one workspace

The native Qt operator places dense graphs and investigation panels inside a managed desktop workspace. The watchdog hosts retained service and tool surfaces, connecting Gitea, BloodHound, build workflows, RDP and development harnesses with the operator environment.



Actual Qt capture - 5 September 2026 - fictional, disconnected fixture. Presentation evidence only.

Lightweight QML chrome surrounds native widgets. Embedded tool windows and custom analysis views can sit alongside each other, preserving tool fidelity while keeping the investigation visible.

Implemented contract

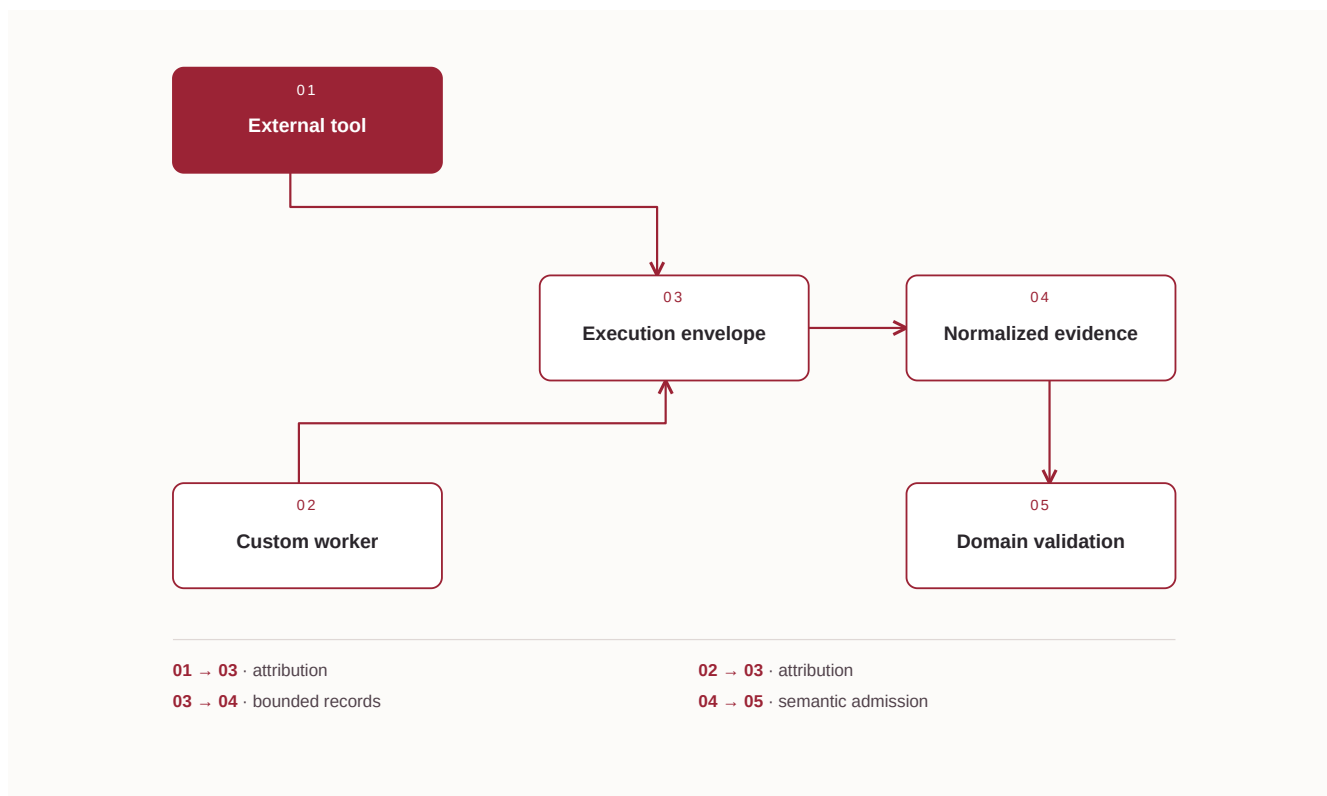
Reference 21

subsystem glossary

WHAT MAKES A TOOL CATALOG MORE THAN A LIST?

Third-party tools, isolated execution

Tool descriptors connect network, web, directory, browser and source-inspection families to the platform. Admitted third-party proof-of-concept work can use isolated container environments, with outputs returned as attributed evidence.



Source-derived conceptual diagram. Arrows describe the labeled relationship.

Integration combines execution policy, process ownership and evidence normalization. Tool availability, candidate review and permission remain distinct inputs to the workflow.

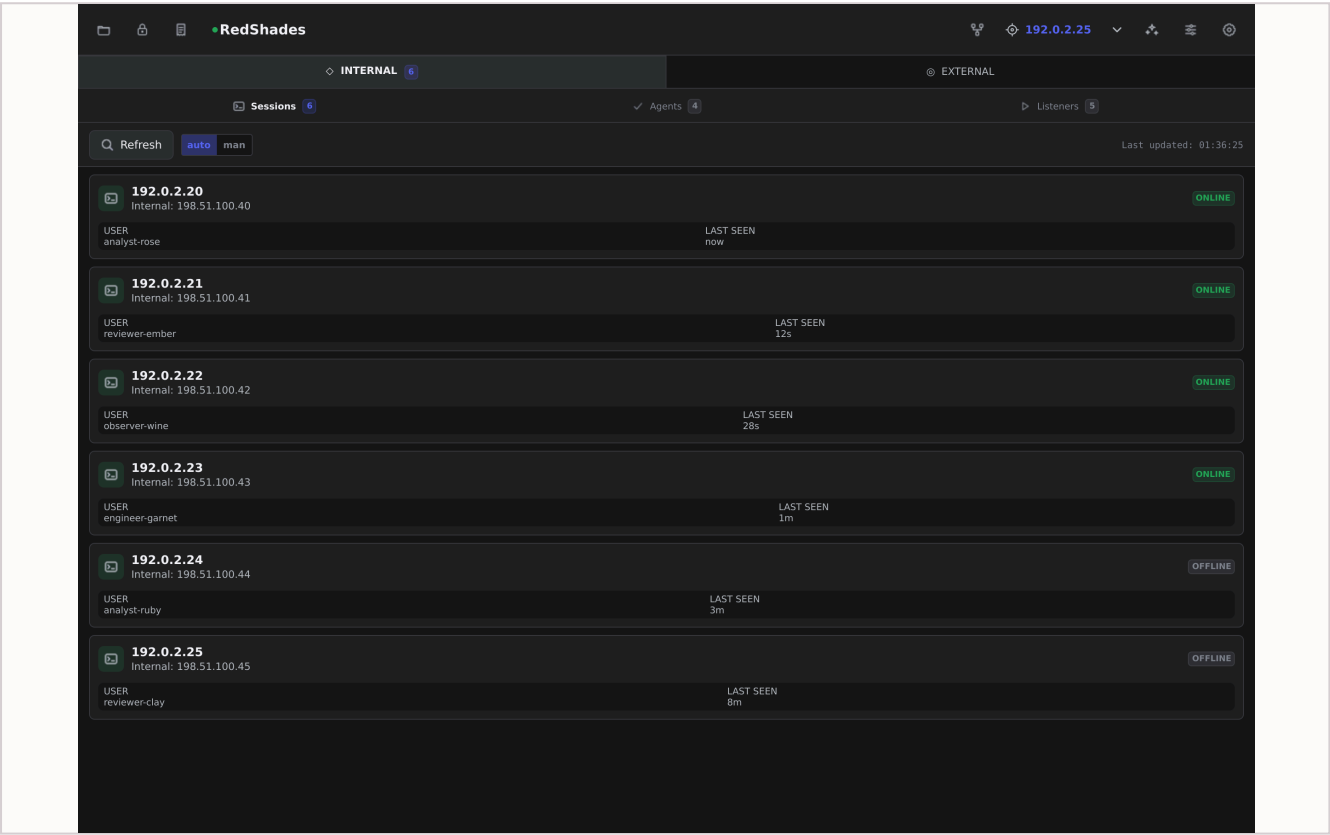
Implemented contract

Reference 22
specialist plugin registry

WHAT BINDS A BINARY TO ITS USE?

VM-based transformation and artifact engineering

The artifact and implant toolchain includes transformation and obfuscation work alongside versioned builds. VM-oriented transformation is part of the project’s engineering breadth, with artifact identity and provenance carried into the wider workspace.



Actual Qt capture - 5 September 2026 - fictional, disconnected fixture. Presentation evidence only.

The publication describes these systems at component level. Build records identify the material and its role; native artifact surfaces keep the resulting inventory accessible to the operator.

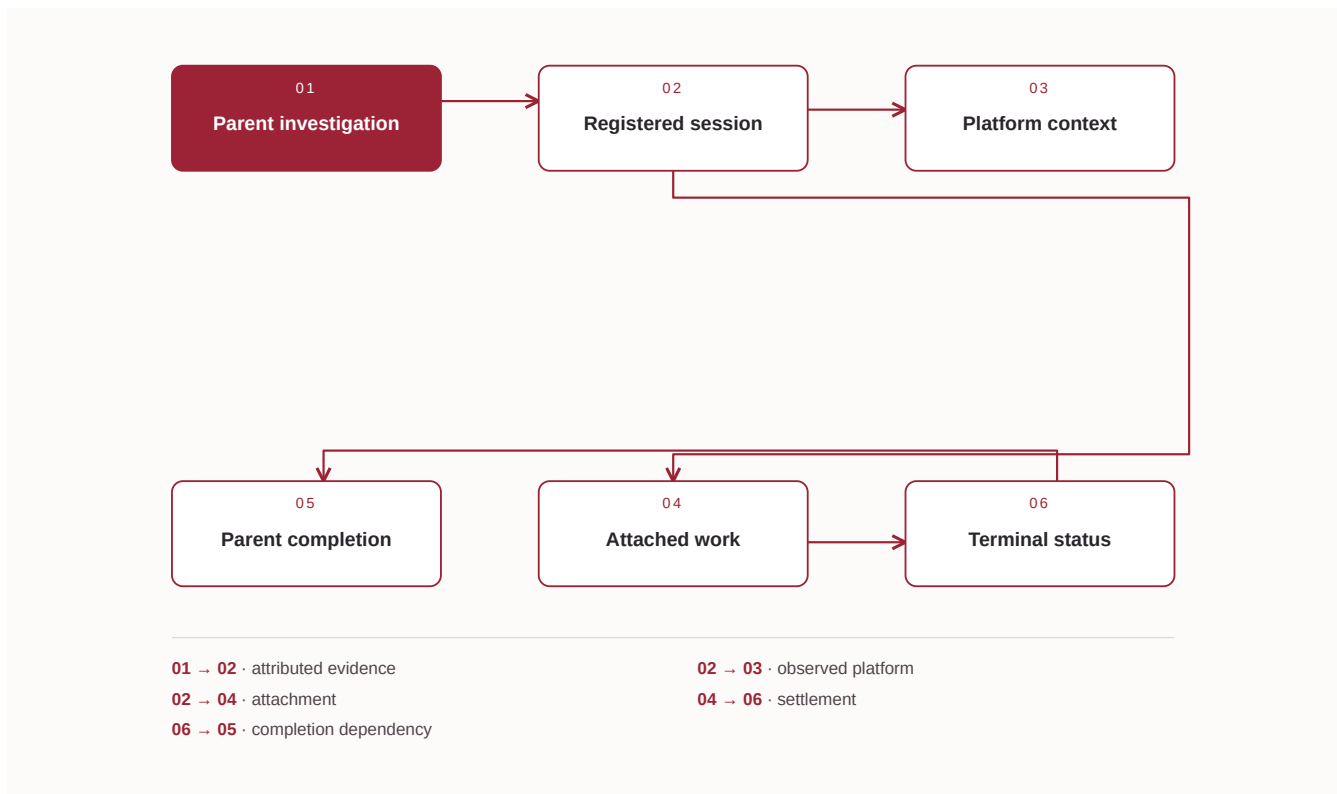
Implemented contract

Reference 23
deployment and process ownership

HOW IS POST-FOOTHOLD WORK REPRESENTED?

Session work extends the investigation

C2 stage families add platform-aware continuation once a session is registered. Context, artifacts and attached work remain connected to the parent engagement, extending the same orchestration model beyond initial observation.



Source-derived conceptual diagram. Arrows describe the labeled relationship.

Common and Windows/Linux declarations preserve their own prerequisites. The automatic stage atlas and the C2 declaration inventory describe separate parts of the runtime graph.

Implemented contract

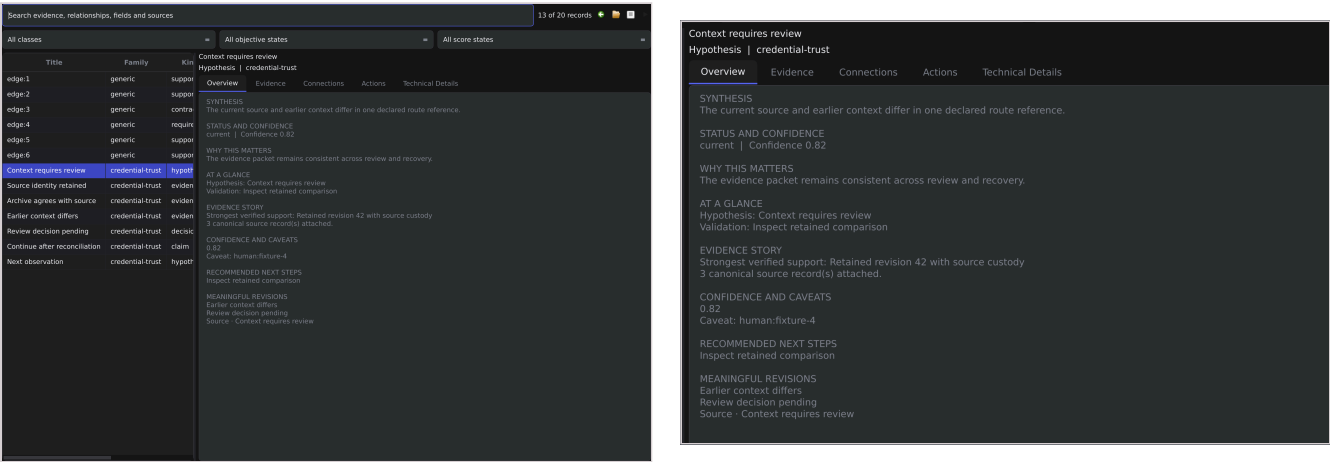
Reference 24

authoritative engagement architecture

EMBEDDED SPECIALIST TOOLS AND CUSTOM ANALYSIS

Reverse engineering inside the workflow

Binary and source analysis connect to the investigation workbench and embedded specialist tools. Custom reverse-engineering workflows can sit beside native evidence views and retained external tool surfaces.



Actual Qt capture - 5 September 2026 - fictional, disconnected fixture. Overview and detail crop from identical source pixels. Presentation evidence only.

The integrated workspace brings analysis, artifact records and engineering context together. Tool-specific interfaces remain available while their results feed the shared investigation record.

Implemented contract

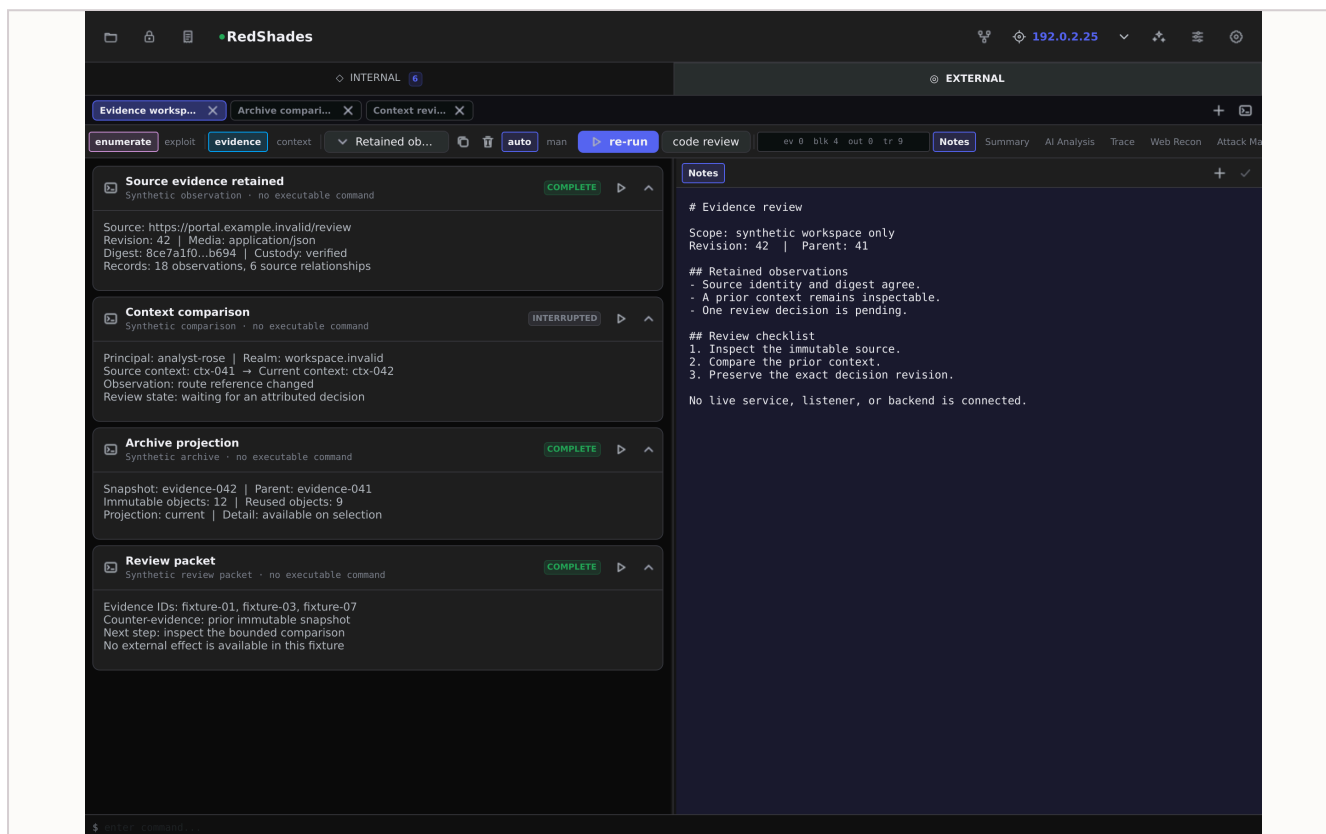
Reference 25

execution graph lifecycle

WHAT SHOULD AN OPERATOR BE ABLE TO EXPLAIN?

A working desktop for the whole investigation

Network Graph, WebRecon, Investigation Graph, session records and model controls share a native Qt operator. Embedded tools extend the workspace rather than forcing every workflow into a generic dashboard.



Actual Qt capture - 5 September 2026 - fictional, disconnected fixture. Presentation evidence only.

The retained screenshots show real Qt widgets with fictional disconnected data. Their arrangement demonstrates how overview, selected evidence and specialist work can remain visible together.

Sanitized native UI evidence

Reference 26

subsystem glossary

NETWORK GRAPH

Read the investigation spatially

The Network Graph presents observed hosts and relationships spatially. Its value is orientation: which context is selected and how that context relates to the wider investigation.



Actual Qt capture - 5 September 2026 - fictional, disconnected fixture. Presentation evidence only.

Native Qt capture with fictional, disconnected demonstration data. The web edition opens the original image at full resolution.

Sanitized native UI evidence

Reference 27

subsystem glossary

WEBRECON OVERVIEW

Keep the source near the decision

WebRecon brings web observations, source detail and endpoint review into a native workspace. A useful arrangement lets the operator retain overview while examining the evidence behind one selected item.



Actual Qt capture - 5 September 2026 - fictional, disconnected fixture. Presentation evidence only.

Native Qt capture with fictional, disconnected demonstration data. The web edition opens the original image at full resolution.

Sanitized native UI evidence

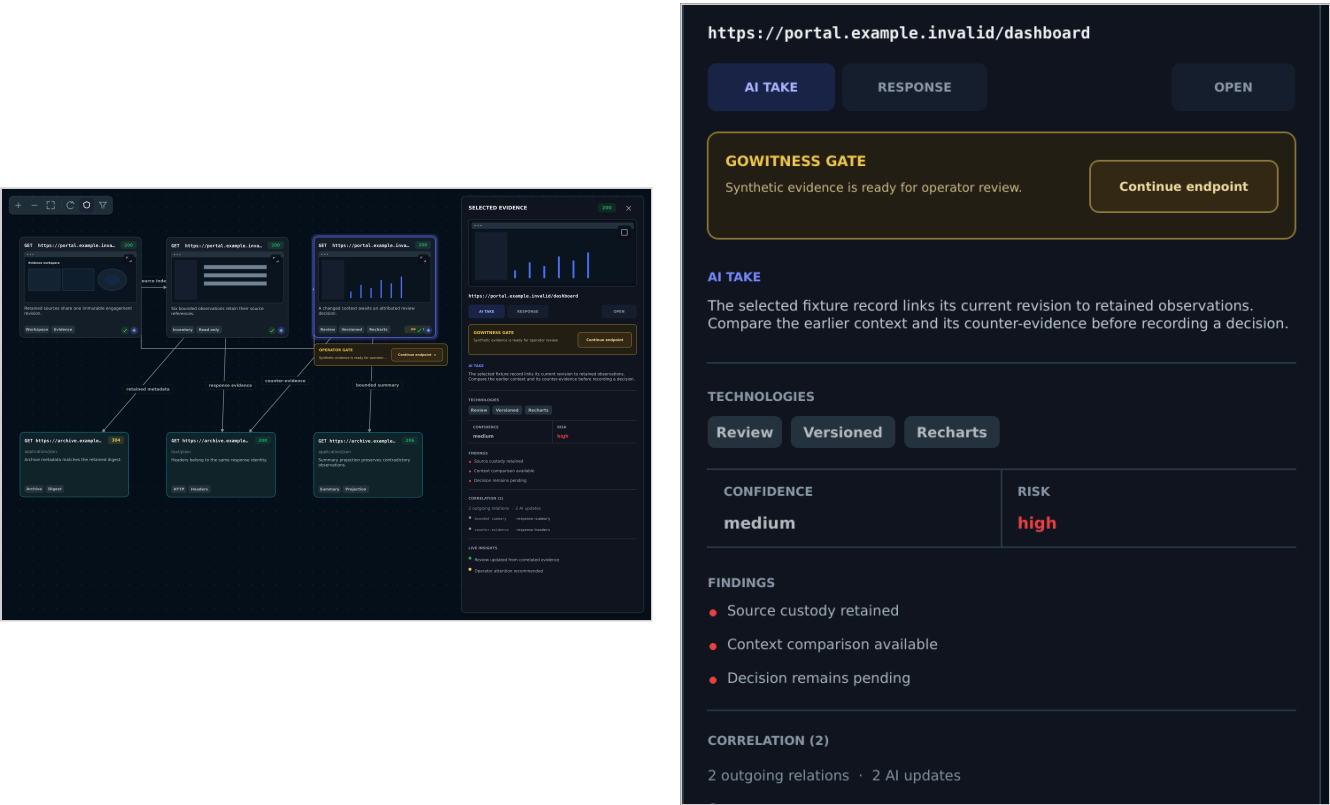
Reference 28

canonical specialist contracts

WEBRECON REVIEW AND GATE STATE

A pending decision has a scope

A gate should explain the affected endpoint or branch and retain the evidence needed for a decision. The rest of the run should not appear globally blocked unless it was explicitly paused.



Actual Qt capture - 5 September 2026 - fictional, disconnected fixture. Overview and detail crop from identical source pixels. Presentation evidence only.

Native Qt capture with fictional, disconnected demonstration data. The web edition opens the original image at full resolution.

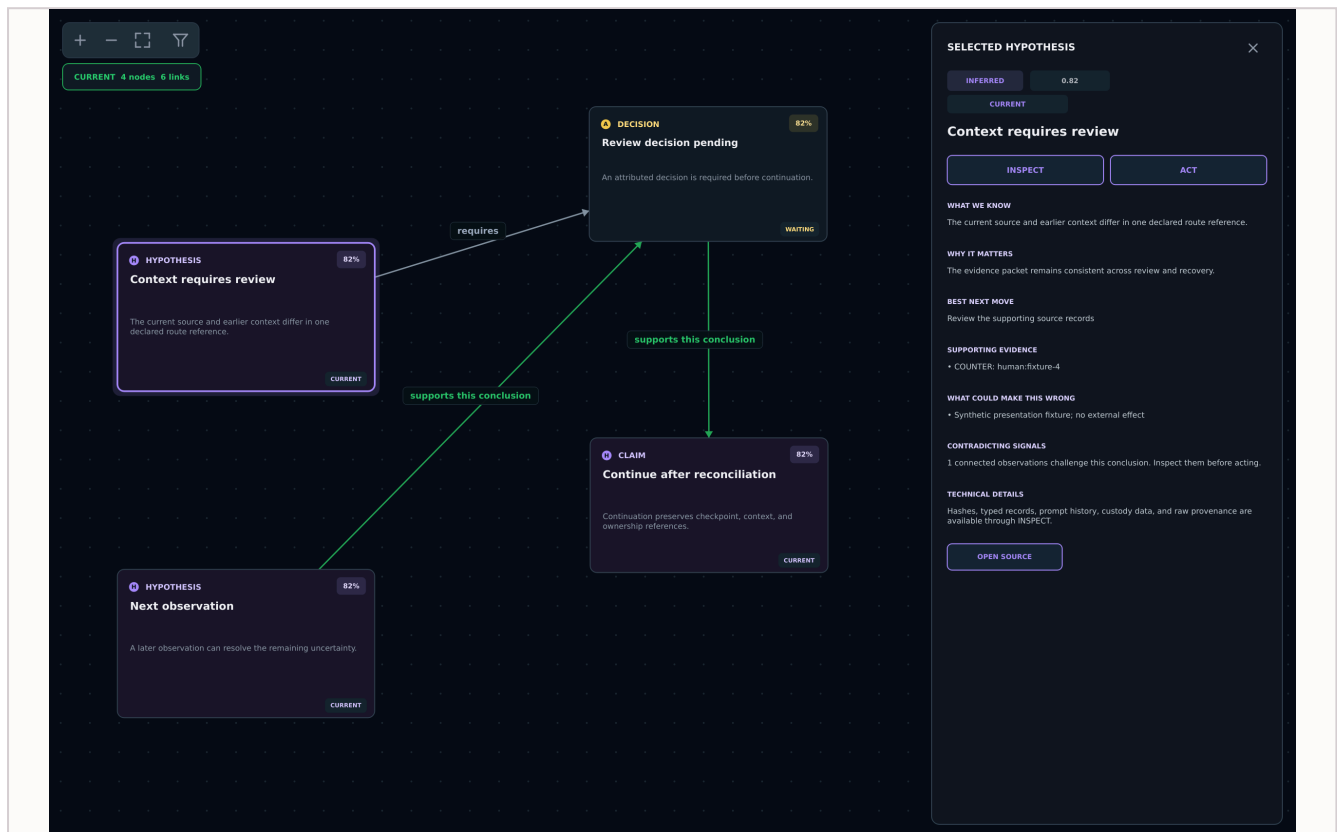
Sanitized native UI evidence

Reference 29
execution graph lifecycle

INVESTIGATION GRAPH

Reasoning needs inspectable sources

Investigation Graph organizes relationships among observations and interpretations. Its companion detail views preserve source references so the operator can inspect the basis of a hypothesis.



Actual Qt capture - 5 September 2026 - fictional, disconnected fixture. Presentation evidence only.

Native Qt capture with fictional, disconnected demonstration data. The web edition opens the original image at full resolution.

Sanitized native UI evidence

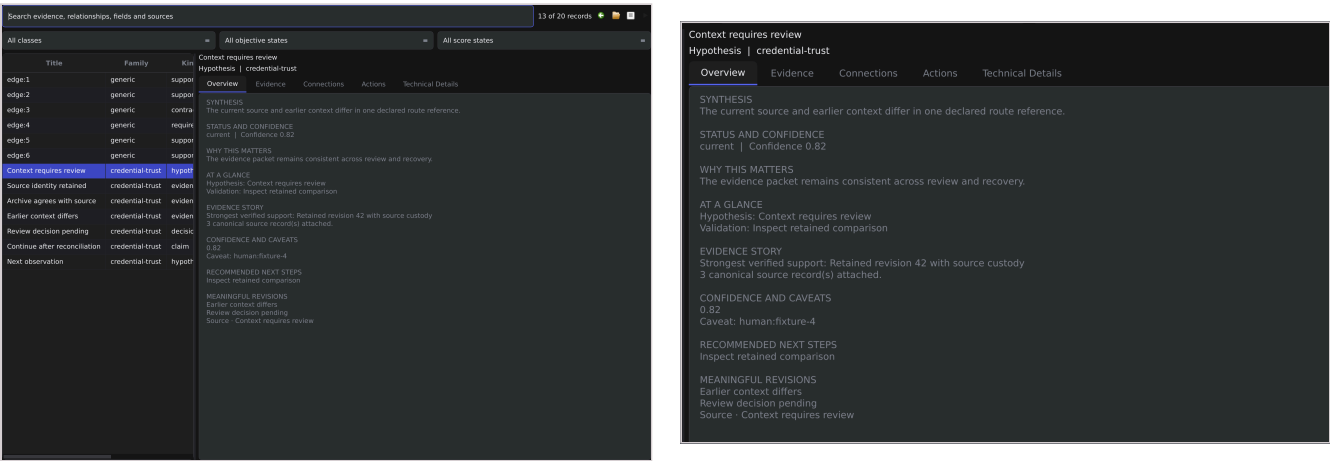
Reference 30

investigation attention

INVESTIGATION WORKBENCH

The detail view carries the difficult questions

The Workbench provides a more deliberate place to inspect evidence, context and available continuation. Dense records benefit from selection and detail rather than forcing every field into the graph itself.



Actual Qt capture - 5 September 2026 - fictional, disconnected fixture. Overview and detail crop from identical source pixels. Presentation evidence only.

Native Qt capture with fictional, disconnected demonstration data. The web edition opens the original image at full resolution.

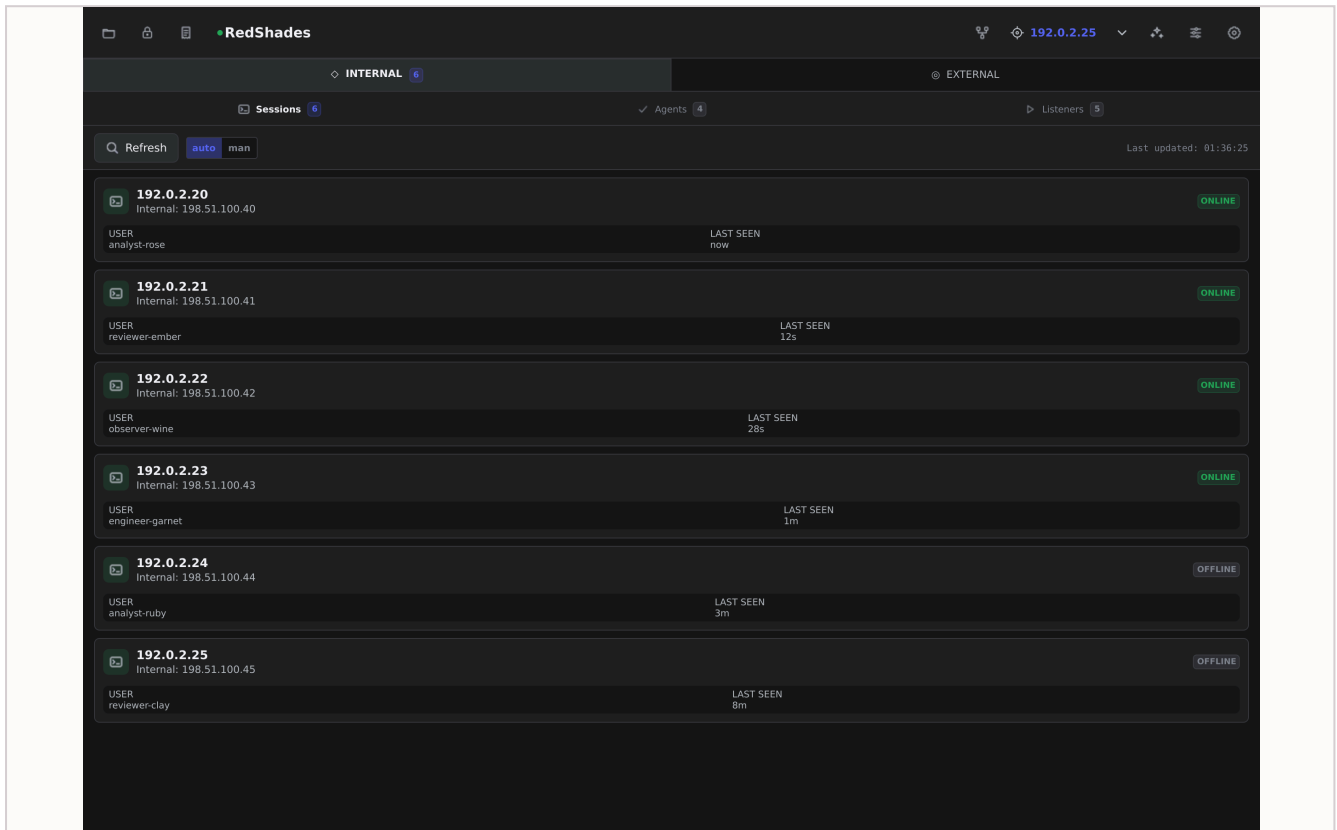
Sanitized native UI evidence

Reference 31
canonical specialist contracts

SESSION AND ARTIFACT SURFACES

Sessions and artifacts at a glance

Session and agent views bring stored records, versioned builds and observed availability into the operator workspace. Selected detail connects the inventory with its context and current state.



Actual Qt capture - 5 September 2026 - fictional, disconnected fixture. Presentation evidence only.

Native Qt capture with fictional, disconnected demonstration data. The web edition opens the original image at full resolution.

Sanitized native UI evidence

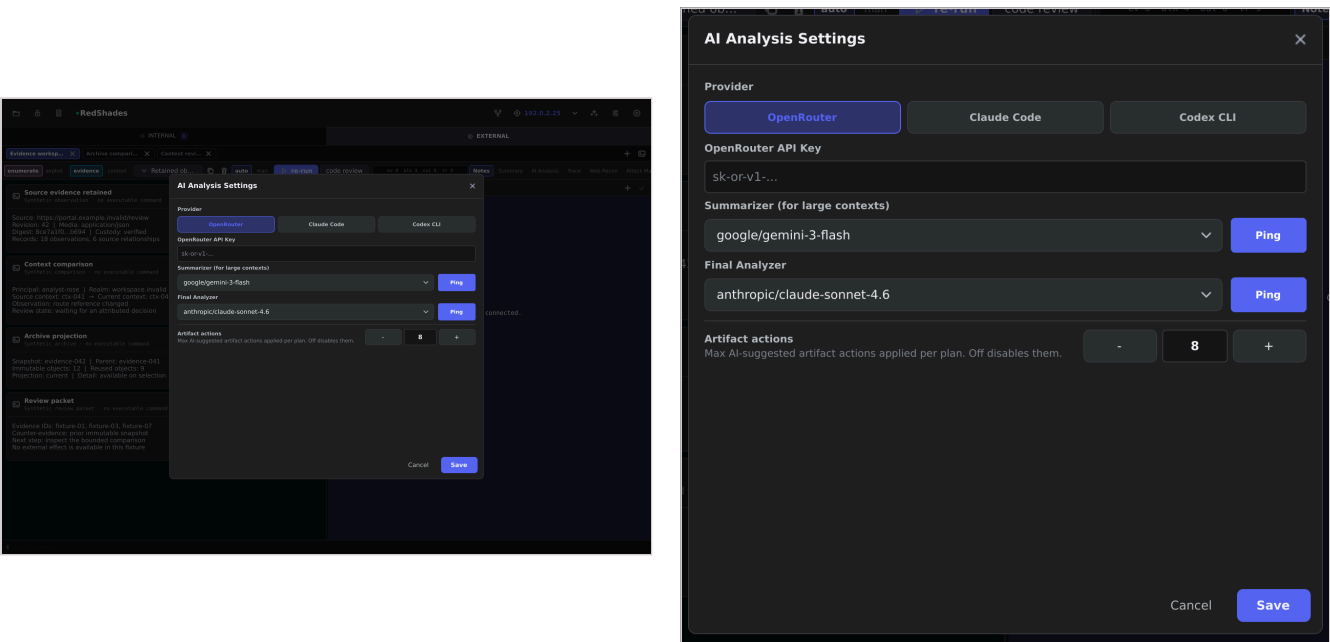
Reference 32

subsystem glossary

MODEL AND RUNTIME CONTROLS

Model and runtime controls in reach

Operator settings expose model and runtime choices in the same workspace as the investigation. Configuration surfaces connect human intent with backend admission, attribution and accounting.



Actual Qt capture - 5 September 2026 - fictional, disconnected fixture. Overview and detail crop from identical source pixels. Presentation evidence only.

Native Qt capture with fictional, disconnected demonstration data. The web edition opens the original image at full resolution.

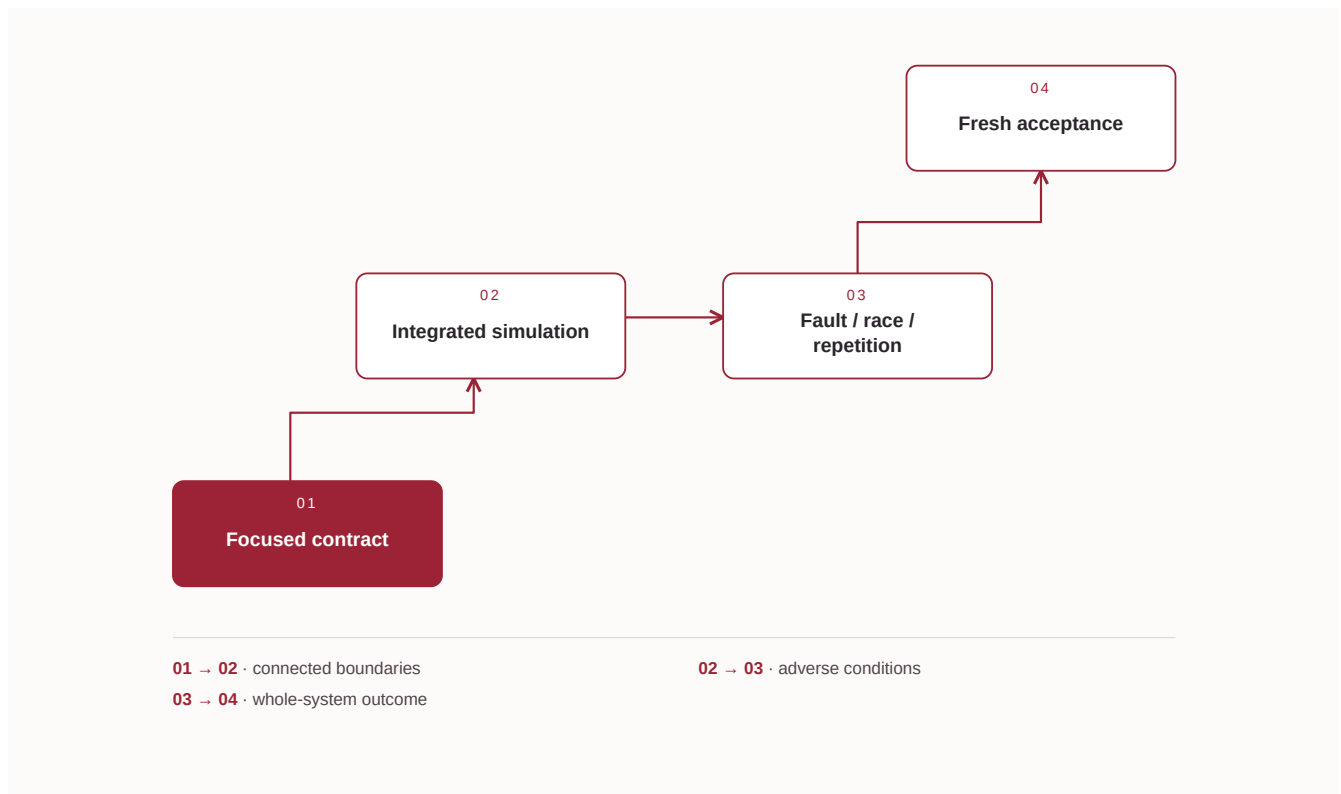
Sanitized native UI evidence

Reference 33
authoritative engagement architecture

HOW SHOULD CONFIDENCE ACCUMULATE?

Verification at every scale

Focused contracts, integrated simulations, race checks, repetition, fuzzing and fault injection exercise different dimensions of the platform. Mega and Ultra organize these into explicit engineering acceptance work.



Source-derived conceptual diagram. Arrows describe the labeled relationship.

Revision-bound records connect each check to its command and output. Validation status is documented separately from the product's architectural capabilities.

Implemented contract

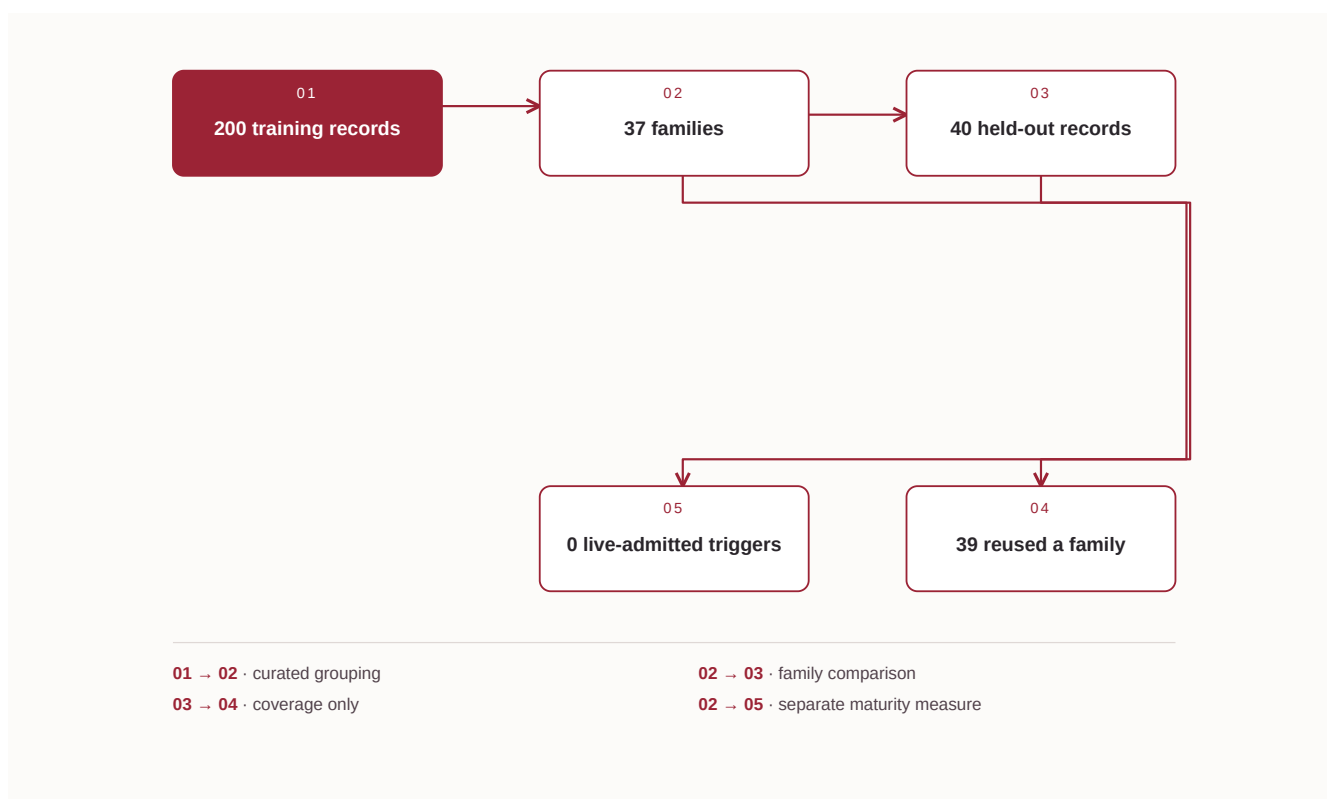
Reference 34

ultra ci requirements

WHAT DID THE DISCLOSURE STUDY MEASURE?

Disclosures become a reusable methodology vocabulary

The Trigger Methodology Graph study organized 200 public disclosures into 37 recurring families. In a 40-disclosure identity-disjoint holdout, 39 belonged to a family represented in that vocabulary.



Source-derived conceptual diagram. Arrows describe the labeled relationship.

This is family coverage in a curated study, not a count of executed discoveries. It motivates evidence-bound hypotheses and controlled follow-up through the diagnostic graph.

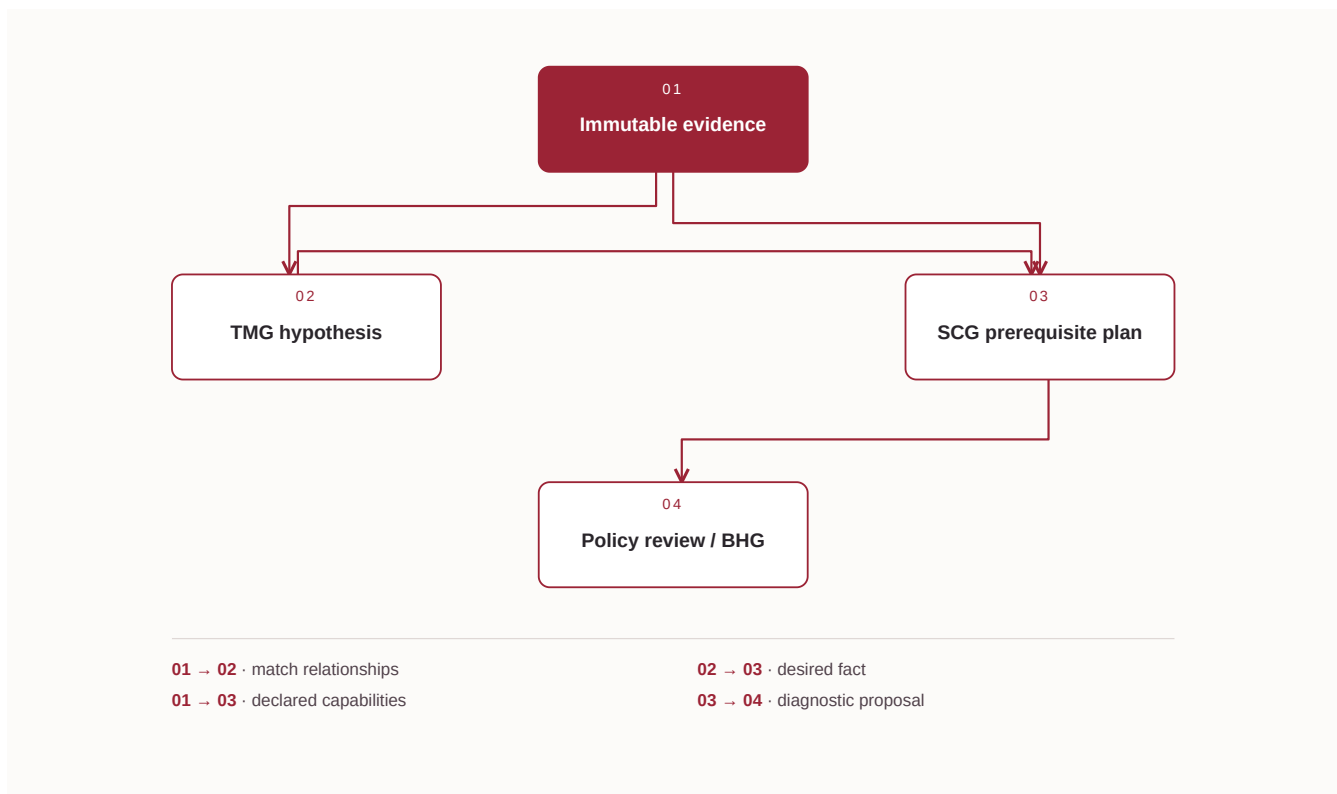
Retained study

Reference 35
tmg disclosure generalization study

HOW DO TMG AND SCG CONTRIBUTE?

A hypothesis should reveal its missing evidence

Trigger Methodology Graph (TMG) produces evidence-bound diagnostic hypotheses. Shadow Capability Graph (SCG) reasons about missing prerequisites from an immutable snapshot. Bounty Hunting Gates (BHG) retain policy review for proposed hypotheses.



Source-derived conceptual diagram. Arrows describe the labeled relationship.

Their useful role is to make uncertainty and the next required evidence explicit. A diagnostic graph does not become scheduling or execution authority simply because its suggestion is plausible.

Implemented contract

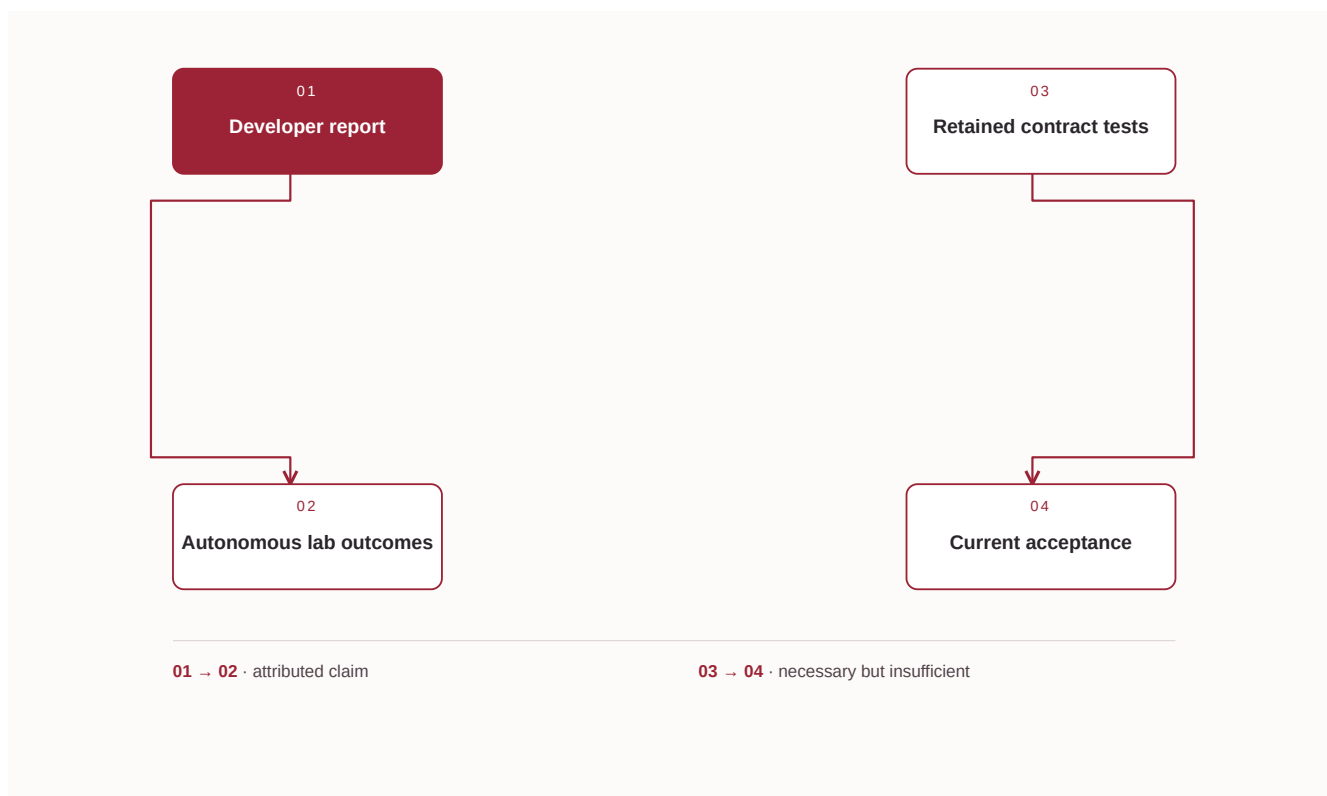
Reference 36

trigger methodology graph + shadow capability graph

WHAT HAS BEEN REPORTED?

Autonomous outcomes across Windows and web labs

The developer reports unattended completion of P.O.O, a hard Windows CTF, an insane Windows HTB lab and roughly two easy Linux/web HTB labs. These outcomes reflect the project's ambition to carry substantial work across an entire engagement.



Source-derived conceptual diagram. Arrows describe the labeled relationship.

Results are owner-reported autonomous outcomes. Independent complete log bundles were not established by this editorial review; the reported runs are not presented as a benchmark for every unseen environment.

Developer-reported autonomous outcomes

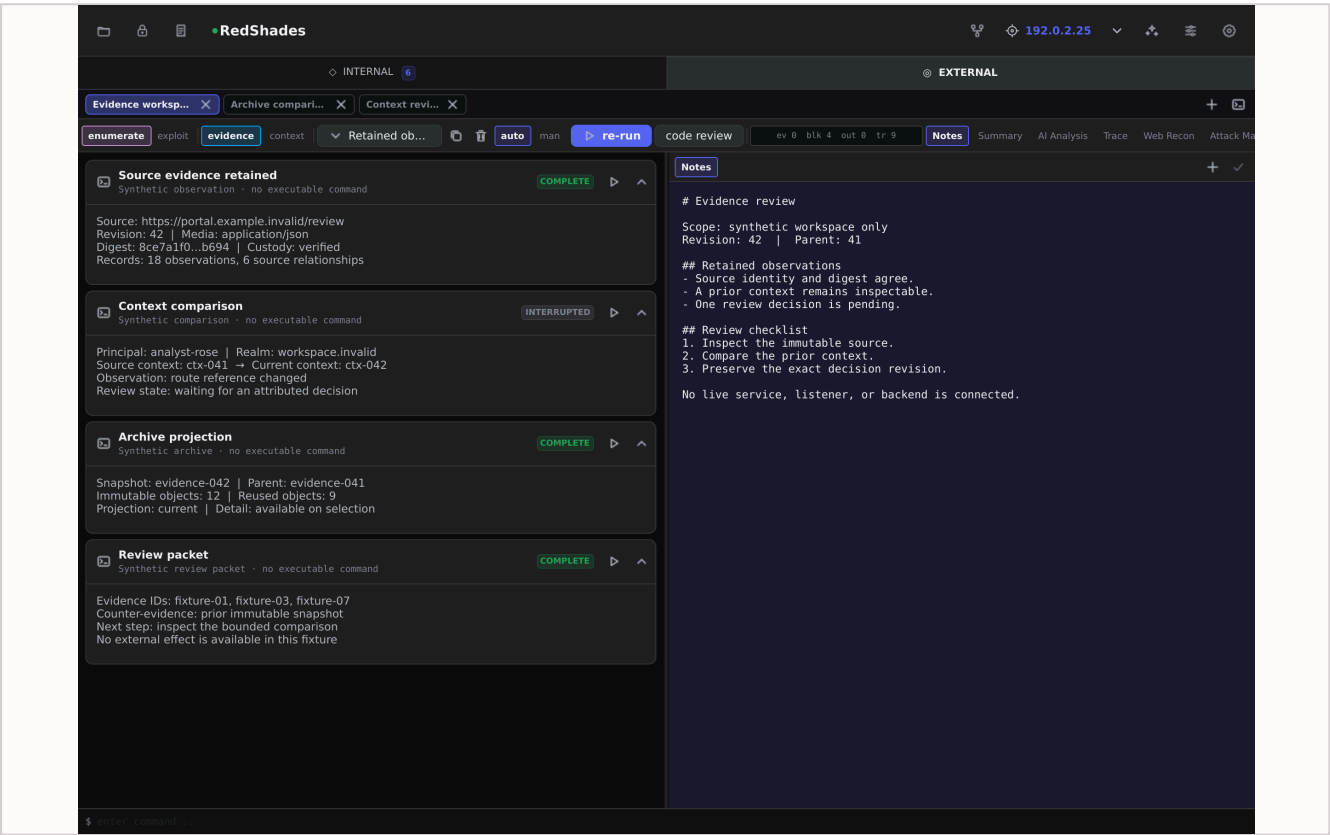
Reference 37

design history

AN INVITATION TO EXAMINE THE BOUNDARIES

Explore the system in a private demonstration

RedShades brings adaptive orchestration, custom capability engineering, runtime repair and a native tool workspace into one product architecture. The investment proposition is the combination: a platform built to make focused model intelligence useful across substantial technical work.



Actual Qt capture - 5 September 2026 - fictional, disconnected fixture. Presentation evidence only.

A private technical demonstration can examine the operator, follow a retained capability through review and explore the engineering behind persistent sessions and hot replacement. Independent technology, presented for technical investors and product leaders.

Technical portfolio

Reference 38
editorial thesis

88 AUTOMATIC DECLARATIONS • CONDITIONAL GRAPH

Stage atlas / 01

Exact source labels and keys. Input and output classes are descriptive summaries; separate C2 families extend session work.

Stage 1: masscan discovery

discovery_masscan

Discovery • Scope and observation context → Network observations

Stage 2: query discovered ports

discovery_ports

Discovery • Scope and observation context → Network observations

Stage 3: nmap service and OS scan

discovery_nmap

Discovery • Scope and observation context → Network observations

Stage 4: service CVE candidate sweep

service_searchsploit_sweep

Service analysis • Service observations → Service evidence

Stage 5: subfinder

service_subfinder

Service analysis • Service observations → Service evidence

Stage 6: passive OSINT (WHOIS / DNS / ASN)

osint_passive

Discovery • Scope and public observations → Context evidence

Stage 7: ssh-audit

service_ssh_audit

Service analysis • Service observations → Service evidence

Stage 8: FTP anonymous

service_ftp_anon

Service analysis • Service observations → Service evidence

Results return to the evidence-driven scheduler. Stage declarations describe available work, not a mandatory linear sequence.

Source-derived inventory

Automatic stages 1–8 / 88

88 AUTOMATIC DECLARATIONS • CONDITIONAL GRAPH

Stage atlas / 02

Exact source labels and keys. Input and output classes are descriptive summaries; separate C2 families extend session work.

Stage 9: NFS enumeration

`service_nfs`

Service analysis • Service observations → Service evidence

Stage 10: Redis enumeration

`service_redis`

Service analysis • Service observations → Service evidence

Stage 11: MySQL probe

`service_mysql`

Service analysis • Service observations → Service evidence

Stage 12: DNS zone transfer drill

`service_dns_axfr`

Service analysis • Service observations → Service evidence

Stage 13: dnsx

`service_dnsx`

Service analysis • Service observations → Service evidence

Stage 14: SMTP user enumeration

`service_smtp_user_enum`

Service analysis • Service observations → Service evidence

Stage 15: SNMP enumeration

`service_snmp_enum`

Service analysis • Service observations → Service evidence

Stage 16: IPMI enumeration and hash extraction

`service_ipmi`

Service analysis • Service observations → Service evidence

Results return to the evidence-driven scheduler. Stage declarations describe available work, not a mandatory linear sequence.

Source-derived inventory

Automatic stages 9–16 / 88

88 AUTOMATIC DECLARATIONS • CONDITIONAL GRAPH

Stage atlas / 03

Exact source labels and keys. Input and output classes are descriptive summaries; separate C2 families extend session work.

Stage 17: service CVE workbench

`service_cve_workbench`

Service analysis • Service observations → Service evidence

Execution context: network path and domain time

`environment_adaptation`

Platform adaptation • Environment context → Adaptation records

Stage 18: enum4linux-ng

`identity_enum4linux`

Identity analysis • Identity and service context → Identity evidence

Stage 19: nxc null/guest enumeration

`identity_nxc_guest`

Identity analysis • Identity and service context → Identity evidence

Stage 20: lookupsid

`identity_lookupsid`

Identity analysis • Identity and service context → Identity evidence

Stage 21: ldapdomaindump

`identity_ldapdomaindump`

Identity analysis • Identity and service context → Identity evidence

Stage 22: LAPS password dump

`identity_laps`

Identity analysis • Identity and service context → Identity evidence

Stage 23: username permutation generation

`identity_username_anarchy`

Identity analysis • Identity and service context → Identity evidence

Results return to the evidence-driven scheduler. Stage declarations describe available work, not a mandatory linear sequence.

Source-derived inventory

Automatic stages 17–24 / 88

88 AUTOMATIC DECLARATIONS • CONDITIONAL GRAPH

Stage atlas / 04

Exact source labels and keys. Input and output classes are descriptive summaries; separate C2 families extend session work.

Stage 24: kerbrute

`identity_kerbrute`

Identity analysis • Identity and service context → Identity evidence

Stage 25: certipy AD CS

`identity_certipy`

Identity analysis • Identity and service context → Identity evidence

Stage 26: GetNPUsers

`identity_getnpusers`

Identity analysis • Identity and service context → Identity evidence

Stage 27: GetUserSPNs

`identity_getuserspns`

Identity analysis • Identity and service context → Identity evidence

Stage 28: AD coercion (PrinterBug/PetitPotam/DFSCoerce)

`identity_coerce`

Identity analysis • Identity and service context → Identity evidence

Stage 29: Responder NTLM hash capture

`identity_ntlm_capture`

Identity analysis • Identity and service context → Identity evidence

Stage 30: NTLM relay (ntlmrelayx)

`identity_ntlm_relay`

Identity analysis • Identity and service context → Identity evidence

Stage 31: hash cracking

`identity_hash_cracking`

Identity analysis • Identity and service context → Identity evidence

Results return to the evidence-driven scheduler. Stage declarations describe available work, not a mandatory linear sequence.

Source-derived inventory

Automatic stages 25–32 / 88

88 AUTOMATIC DECLARATIONS • CONDITIONAL GRAPH

Stage atlas / 05

Exact source labels and keys. Input and output classes are descriptive summaries; separate C2 families extend session work.

Stage 32: targeted credential spray

`identity_credential_spray`

Identity analysis • Identity and service context → Identity evidence

Stage 33: smbmap share enumeration

`remote_smbmap`

Remote services • Service and authorized session context → Protocol evidence

Stage 34: SMB file harvest

`remote_smb_harvest`

Remote services • Service and authorized session context → Protocol evidence

Stage 35: SMB binary credential extraction

`post_smb_binary_analysis`

Extended investigation • Session, artifact or investigation context → Attributed investigation records

Stage 36: conditional smbclient

`remote_smbclient`

Remote services • Service and authorized session context → Protocol evidence

Stage 37: conditional mssqlclient

`remote_mssqlclient`

Remote services • Service and authorized session context → Protocol evidence

Stage 38: MSSQL linked server ADIDNS poisoning

`mssql_linked_server_poison`

Remote services • Database context → Database evidence

Stage 39: certipy AD CS exploitation (ESC1/ESC4/ESC8)

`identity_certipy_exploit`

Identity analysis • Identity and service context → Identity evidence

Results return to the evidence-driven scheduler. Stage declarations describe available work, not a mandatory linear sequence.

Source-derived inventory

Automatic stages 33–40 / 88

88 AUTOMATIC DECLARATIONS • CONDITIONAL GRAPH

Stage atlas / 06

Exact source labels and keys. Input and output classes are descriptive summaries; separate C2 families extend session work.

Kerberos ticket-backed WinRM validation

`remote_kerberos_winrm`

Remote services • Service and authorized session context → Protocol evidence

Stage 40: web fuzzing (vhost+dir+ext)

`web_fuzz`

Web investigation • Web observations and scoped review → Web evidence

Stage 41: web fingerprint + wafw00f

`web_whatweb_waf`

Web investigation • Web observations and scoped review → Web evidence

Stage 42: CMS scanning

`web_cms_scan`

Web investigation • Web observations and scoped review → Web evidence

Stage 43: nuclei parallel

`web_nuclei`

Web investigation • Web observations and scoped review → Web evidence

Stage 44: web content analysis

`web_content_analysis`

Web investigation • Web observations and scoped review → Web evidence

Stage 45: target-specific password generation

`web_cewl`

Web investigation • Web observations and scoped review → Web evidence

Stage 46: CVE PoC workbench review

`web_cve_poc_workbench`

Web investigation • Web observations and scoped review → Web evidence

Results return to the evidence-driven scheduler. Stage declarations describe available work, not a mandatory linear sequence.

Source-derived inventory

Automatic stages 41–48 / 88

88 AUTOMATIC DECLARATIONS · CONDITIONAL GRAPH

Stage atlas / 07

Exact source labels and keys. Input and output classes are descriptive summaries; separate C2 families extend session work.

Stage 47: GoWitness endpoint review

`web_screenshot`

Web investigation · Web observations and scoped review → Web evidence

Stage 48: CMS/app-server RCE (Tomcat/Jenkins/Drupal)

`web_cms_exploit`

Web investigation · Web observations and scoped review → Web evidence

Stage 49: LFI exploitation

`web_lfi_exploit`

Web investigation · Web observations and scoped review → Web evidence

Stage 50: SQLi probe

`web_sql_i_probe`

Web investigation · Web observations and scoped review → Web evidence

Stage 51: XXE probe

`web_xxe_probe`

Web investigation · Web observations and scoped review → Web evidence

Stage 52: IDOR / parameter enumeration

`web_idor_probe`

Web investigation · Web observations and scoped review → Web evidence

Stage 53: HTTP verb tampering

`web_verb_tampering`

Web investigation · Web observations and scoped review → Web evidence

Stage 54: SSTI probe

`web_ssti_probe`

Web investigation · Web observations and scoped review → Web evidence

Results return to the evidence-driven scheduler. Stage declarations describe available work, not a mandatory linear sequence.

Source-derived inventory

Automatic stages 49–56 / 88

88 AUTOMATIC DECLARATIONS • CONDITIONAL GRAPH

Stage atlas / 08

Exact source labels and keys. Input and output classes are descriptive summaries; separate C2 families extend session work.

Stage 55: OS command injection probe

`web_cmdi_probe`

Web investigation · Web observations and scoped review → Web evidence

Stage 56: XSS probe

`web_xss_probe`

Web investigation · Web observations and scoped review → Web evidence

Stage 57: web login brute force

`web_login_bruteforce`

Web investigation · Web observations and scoped review → Web evidence

Stage 58: post-auth surface harvest

`web_post_auth_harvest`

Web investigation · Web observations and scoped review → Web evidence

Stage 59: post-auth exploit probes

`web_post_auth_exploit_probes`

Web investigation · Web observations and scoped review → Web evidence

Stage 60: post-auth CVE PoC workbench review

`web_post_auth_poc_workbench`

Web investigation · Web observations and scoped review → Web evidence

Stage 61: web file upload attack

`web_file_upload_attack`

Web investigation · Web observations and scoped review → Web evidence

Stage 62: SQLi OS shell (--os-cmd / --os-shell)

`web_sql_i_post_exploit`

Web investigation · Web observations and scoped review → Web evidence

Results return to the evidence-driven scheduler. Stage declarations describe available work, not a mandatory linear sequence.

Source-derived inventory

Automatic stages 57–64 / 88

88 AUTOMATIC DECLARATIONS · CONDITIONAL GRAPH

Stage atlas / 09

Exact source labels and keys. Input and output classes are descriptive summaries; separate C2 families extend session work.

Stage 63: web RCE candidate correlation

`web_rce_candidate_correlation`

Web investigation · Web observations and scoped review → Web evidence

Stage 64: web RCE confirmation

`web_rce_confirm`

Web investigation · Web observations and scoped review → Web evidence

Stage 65: web RCE deployment preparation

`web_rce_deployment_prepare`

Web investigation · Web observations and scoped review → Web evidence

Stage 66: credential reuse sweep

`credential_reuse_sweep`

Investigation · Scoped evidence → Attributed records

Stage 67: secretsdump credential harvest

`post_secretsdump`

Extended investigation · Session, artifact or investigation context → Attributed investigation records

Stage 68: DPAPI artifact analysis (operator action)

`post_dpapi`

Extended investigation · Session, artifact or investigation context → Attributed investigation records

Stage 69: pass-the-hash sweep

`post_pth`

Extended investigation · Session, artifact or investigation context → Attributed investigation records

Stage 70: pass-the-ticket (golden/silver ticket)

`post_ptt`

Extended investigation · Session, artifact or investigation context → Attributed investigation records

Results return to the evidence-driven scheduler. Stage declarations describe available work, not a mandatory linear sequence.

Source-derived inventory

Automatic stages 65–72 / 88

88 AUTOMATIC DECLARATIONS • CONDITIONAL GRAPH

Stage atlas / 10

Exact source labels and keys. Input and output classes are descriptive summaries; separate C2 families extend session work.

Stage 71: DCSync domain hash dump

`post_dcsync`

Extended investigation • Session, artifact or investigation context → Attributed investigation records

Stage 72: noPac vulnerability check (CVE-2021-42278/42287)

`post_nopac`

Extended investigation • Session, artifact or investigation context → Attributed investigation records

Cross-realm Kerberos service-ticket custody

`ad_cross_realm_kerberos`

Directory analysis • Directory evidence → Directory analysis records

Stage 73: BloodHound collection

`bloodhound_collection`

Directory analysis • Directory context → Graph analysis material

AD effective capability discovery

`ad_capability_discovery`

Directory analysis • Directory evidence → Directory analysis records

Stage 74: BloodHound ACL attack path analysis

`post_acl_analysis`

Extended investigation • Session, artifact or investigation context → Attributed investigation records

AD compositional capability planning

`ad_capability_planning`

Directory analysis • Directory evidence → Directory analysis records

Stage 75: credential to shell attempts

`shell_attempts`

Session work • Authorized execution context → Session-related records

Results return to the evidence-driven scheduler. Stage declarations describe available work, not a mandatory linear sequence.

Source-derived inventory

Automatic stages 73–80 / 88

88 AUTOMATIC DECLARATIONS • CONDITIONAL GRAPH

Stage atlas / 11

Exact source labels and keys. Input and output classes are descriptive summaries; separate C2 families extend session work.

Stage 76: Windows privilege escalation enumeration

`post_windows_privesc`

Extended investigation • Session, artifact or investigation context → Attributed investigation records

Stage 77: Linux privilege escalation enumeration

`post_linux_privesc`

Extended investigation • Session, artifact or investigation context → Attributed investigation records

Stage 78: xfreerdp RDP session

`rdp_session`

Session work • Authorized session context → Session records

Stage 79: implant deployment

`implant_deployment`

Artifact and session work • Admitted artifact and authorized context → Artifact/session records

Stage 80: chisel SOCKS5 reverse pivot

`post_pivot_chisel`

Extended investigation • Session, artifact or investigation context → Attributed investigation records

Stage 81: deterministic operator report

`summary`

Reporting • Retained investigation evidence → Report projection

Stage 82: Final assessment report

`post_pwndoc_export`

Extended investigation • Session, artifact or investigation context → Attributed investigation records

Stage 83: ACL operator review (post-summary)

`acl_operator_action`

Directory analysis • Directory evidence and operator review → Review record

Results return to the evidence-driven scheduler. Stage declarations describe available work, not a mandatory linear sequence.

Source-derived inventory

Automatic stages 81–88 / 88